zkSketch: Verifiable and Privacy-Preserving Sketch-Based Analytics via Non-Interactive Zero-Knowledge Proofs

Anonymous Author(s)

Abstract

Regulated analytical workloads — periodic financial reporting, blockchain analytics, telemetry archives — publish aggregate answers (top-k, distinct counts, frequencies, quantiles) over privately committed data streams. Approximate sketches (Count-Min, HyperLogLog, SpaceSaving, Greenwald–Khanna) are the standard plaintext tool, and non-interactive zero-knowledge proofs over commitments are the natural cryptographic primitive for verifying answers without disclosing inputs. The two combine poorly under streaming workloads, because regenerating a ZK proof over the entire historical stream after every new committed batch defeats the central appeal of sketches: incremental maintenance.

We present zkSketch, the first non-interactive ZK framework for *streaming sketch maintenance*. zkSketch maintains a privately committed sketch alongside an efficiently updatable proof object: each new batch advances the committed state under a self-contained *transition proof* local to the batch and the previous committed state; per-epoch proofs are composed online via recursive-SNARK composition into a single history root checkable in $O(1)$ work, under three composition topologies (linear chain, balanced binary tree, streaming forest with lazy / eager / smoothed / query-burst peak-chain policies) with explicit trade-offs against batch size, parallelism, and per-core memory; and each transition circuit certifies the batch's *net effect* on the committed sketch via a bounded *batch-effect certificate* — a sortedness-and-segment-aggregation sparse delta for additive sketches, a sparse max-rebind for non-additive Pattern A sketches, or a bounded-state replacement for Pattern B sketches — rather than replaying the sketch's plaintext random-write procedure inside the circuit. The framework instantiates on four canonical sketch families (Count-Min, HyperLogLog, SpaceSaving, Greenwald–Khanna) through a seven-slot interface, with per-family circuit-level optimisations — hash-once-per-distinct-key, wide-output two-field hashing, sparse-rebind commitment, and single-round threshold retention — that target each family's dominant cost line.

Keywords

zero-knowledge proofs, approximate query processing, sketches, verifiable databases, streaming, recursive SNARKs

1 Introduction

Setting. Regulated analytical workloads publish aggregate answers over privately committed data streams. Periodic financial filings report top-k counterparties and distinct-account counts over a committed transaction log; blockchain-analytics indexes report heavy hitters and frequency aggregates over committed blocks; telemetry archives report quantiles and cardinalities over vendor-signed event streams. The raw records are sensitive and cannot be shown to third-party auditors, but the published aggregates must still be verifiable end-to-end against the committed inputs. Compact summaries — Count-Min [8], HyperLogLog [10], SpaceSaving [24], Greenwald–Khanna [14] — are the plaintext tool of choice for the aggregates, and non-interactive zero-knowledge proofs over committed data [2, 16, 21] are the natural cryptographic primitive for verifying answers without disclosing the inputs.

Central challenge: incremental, not from scratch. The central challenge in verifiable sketch analytics is *not* merely to prove a single sketch computation in ZK. Real analytical streams evolve continuously: new records arrive over time, the committed sketch state changes, and queries may be issued before the stream ends. Recomputing a ZK proof over the entire historical stream after every new batch would destroy the central appeal of sketches — their incremental nature in the plaintext world. We therefore ask:

> *Can a single untrusted prover maintain a privately committed sketch together with an efficiently updatable proof object, so that each new batch only requires a local transition proof, while auditors can still verify the entire committed history succinctly at any time?*

Three technical sub-problems. This framing crystallises into three concrete sub-problems that organise the rest of the paper.

(C1) Incremental verifiability. After epoch B_t arrives, the prover should produce a proof for the *transition* $(S_{t-1}, B_t) \to S_t$, not a proof that $S_t = A_\tau(B_1, \ldots, B_t)$ rebuilt from the empty initial state. Update-local proving means the per-batch prover cost depends mainly on the new batch and the changed sketch cells, not on the full history length.

(C2) Succinct history binding. Even with per-batch transition proofs $\pi_1, \ldots, \pi_t$ in hand, a verifier should not be required to check t proofs every time it audits. The system needs an updatable history accumulator that absorbs each new π_t and exposes one compact root commitment $R^{(t)}$ checkable in work independent of t. Recursive SNARKs, folding accumulators, and proof trees are the candidate primitives.

(C3) Efficient transition certification. The per-batch transition circuit must itself be cheap. A direct in-circuit replay of the plaintext sketch update procedure is expensive: sketch updates evaluate data-dependent hashes, write to private addresses determined by those hashes, handle within-batch collisions, and rebind a committed state after each write — the worst-case operation pattern for arithmetic-circuit backends. The transition circuit must therefore avoid replaying the procedure and instead certify only the batch's *net effect* on the committed state.

Two further concerns shape the same design space and are addressed jointly throughout the paper: *(C4) version-correct query answering* — a query proof must be tied to the committed sketch version containing exactly the uploaded prefix visible before the query — and *(C5) leakage control* — public artefacts such as the circuit shape and the committed capacities must not reveal private distributional information such as touched-cell counts or key skew.

Our approach. zkSketch realises the framing above in three layers.

Per-epoch transition proofs (§4, addressing C1 and C3) certify how one committed batch advances the committed sketch state. The transition circuit verifies a bounded *batch-effect certificate* describing the net effect of the batch on the committed sketch state — a sparse additive delta for Count-Min, a sparse max-rebind patch for HyperLogLog, or a bounded-state replacement certificate for SpaceSaving and Greenwald–Khanna — instead of replaying the sketch's hash-determined random writes. The certificate's shape depends only on public capacities, so the circuit is data-oblivious.

Recursive history composition (§4.3, addressing C2) absorbs each new per-epoch proof into a single history root commitment $R^{(t)}$. The root proof $\Pi^{(t)}$ is verifiable in $O(1)$ work irrespective of the number of epochs. We develop the composition design space as three topologies — linear chain for line-rate ingest, balanced binary tree for fixed-window audits, and streaming forest (with lazy / eager / smoothed peak-chain policies) for live unbounded streams — and characterise their trade-offs against per-batch latency, prover memory, parallelism, and per-core budget.

Sketch instantiations (§5, addressing C3, C4, and C5) instantiate the framework on the four sketch families through a seven-slot interface (Π_τ) and a two-pattern dichotomy (Pattern A linear / Pattern A max / Pattern B), tie every query proof to a committed sketch version via epoch-closure boundaries, and bound public leakage to the data-independent capacities declared in M.

Contributions.

- **Streaming sketch maintenance framework.** A committed sketch object that advances version-by-version under self-contained per-epoch transition proofs and an updatable history root, bound via recursive-SNARK composition into one root commitment $R^{(t)}$ summarising the verifiable streaming history. Merge proofs for independently maintained shards are a separate first-class operation.
- **Composition design space.** Three composition topologies — linear chain, balanced binary tree, streaming forest (with lazy / eager / smoothed peak-chain policies) — expose one $O(1)$ verifier interface but differ on online latency, per-batch maintenance cost, parallelism, and per-core memory; we characterise their regimes against deployment knobs.
- **Per-batch transition circuits across four sketch families.** A seven-slot interface and a two-pattern dichotomy instantiate Count-Min, HyperLogLog, SpaceSaving, and Greenwald–Khanna under one circuit pipeline organised around a verification-over-computation discipline, with per-family optimisations (hash-once-per-distinct-key, wide-output two-field hashing, sparse-rebind commitment for Pattern A; single-round threshold retention for Pattern B) that target the family-specific dominant cost lines.

Scope. ZK certifies exact adherence to the *declared sketch algorithm*, not exact equality with the true answer over the underlying dataset; approximation error is inherent to sketch theory. Input completeness — that every real-world event reached the sketch — requires an authenticated input log on the data-owner side and is outside the cryptographic scope of zkSketch.

Roadmap. §2 fixes preliminaries. §3 defines the system model, threat model, and committed sketch object. §4 develops the streaming sketch maintenance framework: per-epoch transition proofs, the long-history bottleneck, recursive interval proofs, and three composition topologies. §5 instantiates the framework circuit-by-circuit on Count-Min and Greenwald–Khanna as the two pattern representatives; HyperLogLog and SpaceSaving appear in the appendix. §6 describes the planned evaluation, and §7 discusses related work.

2 Preliminaries

2.1 Zero-Knowledge Proofs (ZKPs)

Zero-knowledge proofs (ZKPs) allow a prover to convince a verifier that a public statement is true without revealing any information beyond the validity of the statement itself. ZKP protocols are commonly classified as either interactive [13] or non-interactive [5]. For certain public-coin interactive protocols [12], the Fiat–Shamir transformation [9] can be used to turn interactive ZKPs into non-interactive ZKPs. In this work, we focus on non-interactive ZKPs, where the proof is consolidated into a single message that can be verified later by any verifier. This makes such proofs particularly suitable for asynchronous settings in which the prover and verifier need not be online at the same time. More specifically, we use *zero-knowledge succinct non-interactive arguments of knowledge* (ZK-SNARKs) [5], which produce short proofs and support fast verification properties that are well suited to the offline-checkable model. A ZK-SNARK provides three standard guarantees:

- *Completeness*: an honest prover can convince an honest verifier of any true statement.
- *Knowledge soundness*: a dishonest prover cannot convince a verifier of a false statement except with negligible probability, and an extractor recovers the witness from any successful prover.
- *Zero-knowledge*: the verifier learns nothing about the witness beyond the validity of the statement.

At a high level, a ZK-SNARK proves knowledge of a private witness that satisfies a publicly specified relation for a given public statement. The witness contains the hidden information needed to justify why the statement is true; depending on the application, it may include secret inputs, intermediate values, or an execution trace of the computation being certified. To enable efficient proving and verification, the target computation is encoded as an arithmetic circuit, or equivalently as a system of algebraic constraints over a finite field. This circuit is public and specifies how the public inputs and private witness must relate. In practice, the computation is represented by constraints, often expressed as low-degree polynomial equations, that are satisfied exactly when the witness corresponds to a valid execution of the computation. The prover then generates a succinct proof that it knows a witness satisfying all of these constraints, without revealing the witness itself.

Shall we introduce ZKSNARK and ZKSTARK here?

3 System Model and Problem Statement

This section fixes the contract that the rest of the paper implements. §3.1 models the streaming workload, the epoch grouping, and the version-closure rule that anchors every query to a unique closed

prefix. §3.2 walks the per-epoch operational flow that the framework of §4 and the circuits of §5 jointly realise. §3.3 defines the committed sketch object and the public log that the verifier reads. §3.4 names the actors, their trust status, the enumerated threats they may mount, and the cryptographic primitives we assume. §3.5 states the correctness and security properties of zkSketch. §3.6 delimits the protection scope, the public leakage profile, and the efficiency target.

3.1 Streaming Workload and Versions

A *stream* is an interleaving of upload events and query events,

$$\mathcal{X} = \big((o_1, x_1), (o_2, x_2), \ldots\big), \qquad o_j \in \{\text{upload}, \text{query}\}.$$

An *upload* ($o_j = \text{upload}$, x_j) carries a private record x_j that the data owner contributes to the sketched dataset; a *query* ($o_j = \text{query}$, x_j) carries a public query parameter $x_j = q$, such as a key for point-frequency estimation, a rank value for quantile queries, or $\perp$ for parameter-free queries (e.g. cardinality). Two indices co-exist and should not be conflated: j is the *global logical event index* that runs over both uploads and queries, while r is the *upload ordinal* that runs only over the upload subsequence. We write $u_1, u_2, \ldots$ (indexed by r) for the subsequence of uploaded records in their logical order, and a data owner whose upload occurs at logical time j receives the next upload ordinal r and publishes the corresponding commitment C_{u_r}. The epoch grouping and upload-log slice indexing below use r; query-prefix soundness reasoning uses j. Operationally, the public upload log ULog and the public query log QueryLog introduced in §3.2 are two views of a single append-only event log: every upload commitment and every query request carries a globally ordered logical index j, assigned synchronously at arrival and not modifiable afterwards. The verifier uses these indices to determine which upload commitments are strictly before a query, so the prover cannot retroactively reassign a query's index after seeing later uploads.

Epochs. For proof generation, zkSketch groups consecutive uploads into *epochs*

$$B_t = (u_{\ell_{t-1}+1}, \ldots, u_{\ell_t}), \qquad 0 = \ell_0 < \ell_1 < \cdots,$$

indexed by a monotone *sketch version* counter $t \in \mathbb{N}$. Epoch boundaries are a systems knob: they may be chosen by record count, wall-clock time, memory pressure, or query arrival. The special case $|B_t| = 1$ gives item-wise proofs; larger epochs amortize commitment openings, sorting, hash checks, aggregation, and proof generation.

Versioned state. For a sketch family τ, let A_τ be the declared sketch algorithm. Starting from the public empty state S_0, the versioned state after epoch t is

$$S_t = A_\tau.\mathsf{UpdateBatch}(S_{t-1}, B_t),$$

where A_τ.UpdateBatch applies the ordinary item-update rule of A_τ to all records in B_t, in order.

Logical time versus sketch version. We distinguish two indices: *logical time* $j \in \mathbb{N}$ counts operations in the stream $\mathcal{X}$ as observed; the *sketch version* $t \in \mathbb{N}$ counts closed epochs. Epoch closure induces a monotone, non-decreasing map $\mathrm{ver} : \mathbb{N} \to \mathbb{N}$ that sends a logical time j to the smallest closed version whose epochs contain exactly all uploads strictly before j. For a query at logical time j, the intended decoded sketch value is

$$dec = A_\tau.\mathsf{Decode}_{x_j}\big(S_{\mathrm{ver}(j)}\big),$$

and the published answer $ans = policy(dec; \rho)$ is obtained by applying the release policy of §3.3.

Invariant 1 (Version closure). Every query proof is bound to a fully closed version: if a query arrives while an epoch is open, the prover first closes that epoch, commits to it, and proves the corresponding transition before emitting the query proof. Equivalently, no query proof witnesses a state S_t for which B_t has uncommitted records. This is the only place where query timing enters the correctness definition, and is the invariant referenced by Property 3 of §3.5.

3.2 System Workflow

One round of zkSketch— from a data-owner upload to a publishable query answer — proceeds in five operational steps; Figure 1 depicts the corresponding architecture.

Step 1: Upload commitment. Each data owner whose upload arrives at logical time j receives the next upload ordinal r and publishes a binding commitment C_{u_r} to its upload u_r on the public *upload log* $\mathsf{ULog} = (C_{u_1}, C_{u_2}, \ldots)$ — an append-only ordered log of upload commitments indexed by upload ordinal — and privately sends u_r and its opening to the prover over a private channel. This is the only data-owner-side action: after publishing C_{u_r} on ULog and transmitting the opening, the data owner does not participate in proof generation. The ordering in ULog fixes the logical-time arrival sequence of records, so any later attempt to drop or reorder a committed upload is detectable from the public log alone.

Step 2: Epoch closure. When the systems policy fires (record count, wall-clock time, memory pressure, or query arrival, per §3.1), the prover seals the currently open epoch B_t, publishes the public epoch descriptor $D_t = (t, \ell_{t-1} + 1, \ell_t, H^U_{[\ell_{t-1}+1, \ell_t]})$ declaring the contiguous upload-log slice the epoch covers, publishes an epoch-level commitment C_{B_t} to the private epoch payload and that slice $(C_{u_{\ell_{t-1}+1}}, \ldots, C_{u_{\ell_t}})$ of ULog, and publishes the post-epoch state commitment $\mathsf{Com}(S_t)$. The records and their openings remain private witnesses; only the commitments are public. Thus ULog fixes the upload order while C_{B_t} fixes the batched private contents consumed by the transition circuit. The transition proof of Step 3 checks that each private row consumed from C_{B_t} opens to the corresponding upload commitment C_{u_r} in the declared contiguous slice $r \in [\ell_{t-1} + 1, \ell_t]$, so the epoch contents cannot diverge from what was previously committed on ULog.

Step 3: Transition proof. The prover emits a non-interactive proof π_t with public input

$$(D_t, \mathsf{Com}(S_{t-1}), C_{B_t}, \mathsf{Com}(S_t), \mathsf{M}),$$

certifying that $\mathsf{Com}(S_t)$ results from applying the declared batch update to openings of $\mathsf{Com}(S_{t-1})$ and C_{B_t} under the public metadata M, and that the private rows consumed from C_{B_t} open to the upload-log slice declared by D_t. The per-family construction of π_t is the subject of §5.

Step 4: History composition. The prover folds π_t into a history root $R^{(t)}$ that summarises the ordered, committed sequence of epochs through version t. The composition mechanics — three topologies (chained, balanced binary tree, streaming forest) and their prover/verifier/latency trade-offs — are the subject of §4.

Step 5: Query answering. When a query arrives at logical time j, its request (j, q) is immediately appended to the public query-request log before any later uploads are processed. The prover then closes any open epoch by Step 2; let $t = \mathrm{ver}(j)$ be the resulting closed version, to which the logged query-request entry is bound. The decoded value *dec* is then published together with a decode proof against $\mathsf{Com}(S_t)$ and, when the release policy applies non-trivial post-processing, the released $ans = policy(dec; \rho)$ together with a release proof certifying it. Query handling and the lazy/eager boundary between Steps 4 and 5 are treated in §4.5.

Asynchrony. Steps 1–4 run online as the stream advances; the answer-and-proof output of Step 5 may be generated offline at any later time, against the closed version recorded at query arrival, using only the public log of §3.3. Logging the query request at arrival is what prevents the prover from retroactively choosing a more convenient version.

Figure placeholder. A left-to-right pipeline. Left: an operation stream with uploads and queries. Middle-left: consecutive uploads are sealed into epochs $B_1, B_2, \ldots$, each with a public commitment C_{B_t}. Middle-right: an untrusted prover maintains $\mathsf{Com}(S_t)$ and $R^{(t)}$. Each epoch emits a transition proof; queries emit decode and optional release proofs against a closed version. Right: an offline verifier checks the public log: epoch commitments, state commitments, history roots, metadata, answers, and proofs.

Figure 1: System model of zkSketch. Uploads are grouped into committed epochs, each epoch advances a committed sketch state, and queries are answered only against closed versions. The verifier checks public commitments, roots, answers, and proofs offline. The figure intentionally abstracts the per-epoch fold of π_t into $R^{(t)}$; the three composition topologies that realise this fold are the subject of §4.

3.3 Committed Sketch Object and Public Log

For each sketch instance, zkSketch maintains a *committed sketch object* at version t,

$$\mathsf{Obj}_t = (id, \tau, params, \mathsf{M}, t, \mathsf{Com}(S_t), R^{(t)}, policy),$$

whose components are: the **instance identifier** *id*; the **family tag** $\tau \in \{\mathsf{CM}, \mathsf{HLL}, \mathsf{SS}, \mathsf{GK}\}$; the **public accuracy and capacity parameters** *params* (the Count-Min dimensions (d, w), the HyperLogLog register count, the SpaceSaving capacity, or the GK capacity parameter); the **public metadata** M, fixing all algorithm choices required to check proofs (hash profile, counter representation, commitment scheme, circuit capacities, proof-system parameters); the **version counter** t; the **state commitment** $\mathsf{Com}(S_t)$, which binds the hidden sketch state at version t; the **history root** $R^{(t)}$, the *canonical* root digest for version t that binds the ordered sequence of committed epochs and state transitions up to that version (under lazy and smoothed policies $R^{(t)}$ may be computed and published on the public log only when $t \in \mathcal{A}_t$; under eager it is materialised at every version); and the **release policy** *policy*, which specifies deterministic post-processing of decoded answers (e.g. rounding, suppression below a threshold, noise generation from a committed seed). The two commitments play different roles: $\mathsf{Com}(S_t)$ says *what* the current private sketch state is, while $R^{(t)}$ says *why* that state is the result of the committed stream prefix. Section 4 constructs $R^{(t)}$ using recursive history composition. Whether the companion root proof $\Pi^{(t)}$ accompanies $R^{(t)}$ at every version t is policy-dependent: the eager peak-chain policy of §4.4 materialises $(R^{(t)}, \Pi^{(t)})$ at every closed version, while the lazy and smoothed policies materialise the pair only for versions $s \in \mathcal{A}_t$ on the audited set introduced below; the verifier reads the pair from $\mathsf{Log}^{(t)}$ when $\mathcal{A}_t$ contains the version it asks about.

The *public log* at version t is the ordered tuple

$$\mathsf{Log}^{(t)} = \Big(\mathsf{ULog}^{(t)},\ \big((D_i,\ C_{B_i},\ \mathsf{Com}(S_i),\ \pi_i)\big)_{i=1}^{t},\ \big((R^{(s)},\ \Pi^{(s)})\big)_{s\in\mathcal{A}_t},\ \mathsf{QueryLog}^{(t)}\Big)$$

containing the prefix $\mathsf{ULog}^{(t)}$ of upload commitments published through version t; for each closed epoch $i \le t$, a public *epoch descriptor* $D_i = (i,\ \ell_{i-1} + 1,\ \ell_i,\ H^U_{[\ell_{i-1}+1,\ell_i]})$ that records the epoch index, the contiguous upload-log range it claims, and a collision-resistant digest $H^U_{[\ell_{i-1}+1,\ell_i]}$ of the upload-commitment slice it covers, together with the binding epoch commitment C_{B_i}, the post-epoch state commitment $\mathsf{Com}(S_i)$, and the per-epoch transition proof π_i; the history-root digest $R^{(s)}$ and root proof $\Pi^{(s)}$ for every version $s \in \mathcal{A}_t$ whose root has actually been materialised; and the query log $\mathsf{QueryLog}^{(t)}$ defined below. The descriptor D_i is a public input to π_i and is what makes the contiguous-partition requirement of Property 2 directly auditable from the public log: a verifier checks that the declared ranges $(D_1, D_2, \ldots, D_t)$ tile the upload-log prefix without gaps, overlaps, or reordering, and that each $H^U_{[\cdot]}$ matches the corresponding slice of ULog. The set $\mathcal{A}_t$ depends on the maintenance policy of §4.4: under eager, $\mathcal{A}_t = \{1, \ldots, t\}$; under lazy or smoothed, $\mathcal{A}_t$ contains the queried or audited versions whose roots have been finalised. The verifier checks each $\Pi^{(s)}$ against the public boundary whose digest is $R^{(s)}$ — the digest alone is not a proof — and may retain the leaf transition proofs π_i for audit, recovery, or debugging without re-checking them on the normal history-verification path. Because the release policy is public, a release proof certifies that the published answer was derived correctly from the decoded sketch answer.

The query log itself separates three phases: $\mathsf{QueryLog}^{(t)}$ contains *request records* (j, q) logged synchronously at query arrival per Step 5 of §3.2; *binding records* (j, t) written after epoch closure fixes the version $t = \mathrm{ver}(j)$ the query must answer against; and, once answered, *response records* (ans, π_q) plus any release proof. The three-phase split captures the asynchrony of Step 5: the request is logged at logical time j before the version t is known, the binding to t is added after epoch closure, and the answer and proof may be produced later still.

3.4 Actors, Trust Model, and Cryptographic Assumptions

Three roles participate.

Data owners (honest). Data owners generate upload records. An owner whose upload arrives at logical time j receives the next upload ordinal r, publishes the binding commitment C_{u_r} on the public upload log ULog, and privately transmits the record u_r and its opening to the prover over a private channel; the public commitment fixes the upload's order and contents while the private channel delivers the witness the prover needs to extend the sketch. After this dual-write, data owners do not participate in proof generation.

Prover (untrusted). The prover maintains the private sketch state, executes all operational steps of §3.2, and produces all non-interactive proofs. A malicious prover may attempt any of the following threats:

(T1) omit a committed upload from an epoch;
(T2) reorder or replay epochs;
(T3) use sketch parameters or hash functions other than those fixed in M;
(T4) fabricate a state transition that does not arise from the declared batch update;
(T5) answer a query from a stale or not-yet-closed version;
(T6) publish an answer that is not the declared decoder's output on the bound sketch state.

Threats T1–T6 are addressed by the security properties of §3.5; each property names the threats it neutralises.

Verifier (honest-but-curious). The verifier is an auditor, regulator, client, or third party who checks $\mathsf{Log}^{(t)}$ offline. The verifier sees only the public log and never raw records or private sketch states. Because the verifier may be curious, proof artifacts must not reveal private information beyond what is intentionally disclosed; the precise public leakage profile is specified in §3.6.

Assumption 1 (Cryptographic primitives). We assume: *(i) NIZK.* The underlying non-interactive zero-knowledge proof system is complete, knowledge-sound, and zero-knowledge against any probabilistic polynomial-time prover. *(ii) Commitments.* The commitment schemes used for uploads, epochs, and sketch state are computationally binding, and computationally hiding wherever an input or witness must remain secret. *(iii) Hashing.* The hash function used to bind ordered histories is collision-resistant. A representative instantiation uses standard vector commitment schemes (e.g. IPA, KZG, or Pedersen) and a recursive ZK-SNARK over an arithmetic circuit; concrete backend choices are deferred to §5 and the evaluation.

3.5 Problem Statement and Security Properties

For a closed version t, the intended sketch state is the unique state reachable from S_0 by applying the declared batch updates $B_1, \ldots, B_t$ in order, satisfying the per-epoch recurrence

$$S_i = A_\tau.\mathsf{UpdateBatch}(S_{i-1}, B_i).$$

The public commitments $\mathsf{Com}(S_t)$ and $R^{(t)}$ are required to bind this state and its ordered update history. For any query q answered at closed version t, the intended decoded sketch answer is

$$dec = A_\tau.\mathsf{Decode}_q(S_t),$$

and the published answer is

$$ans = policy(dec;\ \rho),$$

where *policy* is the public release policy of §3.3 and ρ is the committed release seed (when *policy* is the identity, $ans = dec$). zkSketch must construct non-interactive proofs for committed sketch maintenance that satisfy the four properties below.

Property 1 (Update soundness). If an epoch-transition proof verifies for public input $(D_t,\ \mathsf{Com}(S_{t-1}),\ C_{B_t},\ \mathsf{Com}(S_t),\ \mathsf{M})$, then $\mathsf{Com}(S_t)$ opens to the state obtained by applying the declared batch update to an opening of $\mathsf{Com}(S_{t-1})$ and an opening of C_{B_t} under M, and the rows consumed from C_{B_t} open to exactly the upload-log slice declared by the epoch descriptor D_t. Any malicious prover that fabricates the transition, alters the epoch contents, or substitutes metadata succeeds with at most negligible probability under Assumption 1. *Addresses T3 and T4; instantiated by the per-epoch transition circuits of §5.*

Property 2 (History soundness). If the root proof for $R^{(t)}$ verifies, then $\mathsf{Com}(S_t)$ is reachable from $\mathsf{Com}(S_0)$ by applying epochs $B_1, \ldots, B_t$ whose declared index ranges form a contiguous partition of the upload-log prefix $(C_{u_1}, \ldots, C_{u_{\ell_t}})$, and whose private rows open to the corresponding upload commitments. Dropping, replacing, or reordering an epoch — or omitting a publicly logged upload commitment from the epoch partition — changes the history root except with negligible probability. *Addresses T1 and T2; instantiated by the recursive history composition of §4.*

Property 3 (Query-prefix soundness). If a decode proof verifies for a query at logical time j, then the decoded answer is exactly $dec = A_\tau.\mathsf{Decode}_q(S_{\mathrm{ver}(j)})$ on the closed sketch version of §3.1 — the state containing exactly all uploads strictly before j — and the published answer is $ans = policy(dec;\ \rho)$ under the declared release policy (with a release proof when *policy* is non-trivial). The prover cannot answer from a stale version, from a state containing uploads that should not yet be visible to the query, or from a decoder other than the one declared in M. *Addresses T5 and T6; rests on Invariant 1; query-time mechanics are treated in §4.5.*

Property 4 (Zero knowledge and circuit obliviousness). Proofs hide all private witnesses, including raw records, sketch states, certificate tables, commitment openings, and intermediate values. In addition, the public circuit shape depends only on declared public capacities and metadata, not on private quantities such as the number of distinct keys, touched cells, fusions, or heavy hitters. *Addresses the curious-verifier threat surface; the public leakage profile and the handling of adaptive capacities are specified in §3.6.*

3.6 Protection Scope and Cost Model

Scope. zkSketch certifies faithful execution of the declared sketch algorithm over the committed input stream. Three classes of concerns lie outside its cryptographic scope. **Approximation error:** zkSketch does not certify that the approximate sketch answer equals the exact answer over the raw dataset; approximation error is inherited from the chosen sketch family and its public accuracy parameters. **Input completeness:** guaranteeing that every real-world

event was actually committed into the stream requires an authenticated input log or a data-owner-side mechanism, which is orthogonal to the cryptographic guarantees of zkSketch. **Application-layer integrity:** selection of the query schedule, of the release policy, and of who is authorised to receive released answers is the responsibility of the deploying application.

Leakage profile. The public log $\mathrm{Log}^{(t)}$ and any released answer reveal exactly the following six classes of information. **Public metadata** M, including the sketch family τ, the algorithm parameters *params*, and the proof-system parameters. **Public capacities,** including sketch dimensions, padded epoch sizes, and the public number of fusion or eviction rounds encoded in circuit shape. **Upload-log metadata,** including the number and public order of upload commitments in ULog, and their publication timing if the deployment exposes timestamps. **Epoch-closure timing,** i.e. the wall-clock pattern of epoch boundaries $\ell_1 < \ell_2 < \cdots$ visible from the publication of successive C_{B_t} on the public log. **Query timing,** i.e. the schedule of values $\mathrm{ver}(j)$ at which queries are issued against closed versions. **Released answers** *ans* and any post-processed release under *policy*. The public capacities are themselves data-independent by declaration in M, so circuit obliviousness composes with the four categories above without leaking additional distributional information.

Cost model. The efficiency target of zkSketch is incremental, history-independent cost. **Per-epoch prover cost** depends primarily on $|B_t|$, the sketch parameters, and the certified per-epoch batch effect (e.g. the number of touched counters), not on the cumulative stream length. **Per-query verifier cost** is a constant number of succinct proof verifications, independent of the cumulative stream length. **Per-query proof size** is constant in the cumulative stream length. The constants in this cost model are instantiated by the composition topology cost surface of §4 and by the per-family circuit constraint counts of §5, and are evaluated empirically in §6.

4 zkSketch Framework: Verifiable Streaming Sketch Maintenance

This section develops the proof-orchestration layer of zkSketch. The stream is potentially unbounded: epochs close over time, queries may arrive before the stream ends, and the prover cannot afford to regenerate a proof from the empty sketch state after every new epoch. Instead, zkSketch uses *incremental maintenance*: each closed epoch B_t produces a local transition proof from $\mathsf{Com}(S_{t-1})$ to $\mathsf{Com}(S_t)$, and a history-composition layer recursively combines these local proofs into a single root object for auditing. The prover maintains proof material online as epochs close, so new proof generation builds on already-proved transitions rather than restarting from the beginning. Depending on the composition topology of §4.3, the current root proof may be ready immediately or may require a small finalisation step at query time; the lazy/eager/smoothed choice is the subject of that subsection.

§4.1 introduces the most direct realisation (one ZKP per closed epoch) and exposes the verifier-side $\Theta(E)$ bottleneck it leaves open. §4.2 collapses the chain of per-epoch proofs into a single root proof via recursive interval proofs, fixing the merge atom and the interval-proof contract. §4.3 schedules these merges over a streaming history as a forest of perfect subtrees, structurally analogous to a Merkle Mountain Range [33] but used only to organise the recursive composition. §4.5 handles queries interleaved with updates. The end-to-end soundness and privacy guarantees of the framework are stated in Appendix A.

4.1 Per-epoch proof chains and their cost

To process the stream, the most direct strategy is to generate one ZK proof per closed epoch. Whenever an epoch B_i is closed, the prover produces a proof π_i certifying that the committed sketch state was advanced from $\mathsf{Com}(S_{i-1})$ to $\mathsf{Com}(S_i)$ by applying the declared batch update to B_i.

We formalise this as a per-epoch transition proof. Each π_i has public input

$$x_i \;=\; \big(D_i,\ \mathsf{Com}(S_{i-1}),\ C_{B_i},\ \mathsf{Com}(S_i),\ \mathsf{M}\big),$$

where D_i is the public epoch descriptor of §3.3 that records the epoch index, its declared upload-log slice $[\ell_{i-1}{+}1, \ell_i]$, and the slice digest $H^{U}_{[\ell_{i-1}+1,\ell_i]}$, and M is the public metadata that fixes the sketch parameters, hash profile, counter representation, commitment scheme, and circuit capacities. The private witness w_i contains the old state, the opening of B_i, the new state, and any auxiliary witness required by the update circuit of Section 5. The proof certifies membership in the per-epoch update relation

$$\mathcal{R}^{\tau}_{\mathsf{upd}} \;=\; \{(x_i, w_i) \;:\; S_i = A_\tau.\mathsf{UpdateBatch}(S_{i-1},\, B_i)\},$$

where τ identifies the sketch family. The item-wise case of §3 is the special case $|B_i| = 1$.

The proof π_i is self-contained: it verifies against x_i alone, without reading past or future epochs, so the prover can emit $\mathsf{Com}(S_i)$ and π_i online as soon as B_i is closed. Consecutive proofs are chained through their boundary state commitments: the next proof π_{i+1} takes $\mathsf{Com}(S_i)$ as its previous-state commitment. We call the resulting sequence the *per-epoch proof chain*.

A verifier that wishes to check all epochs have been correctly applied to the sketch verifies every proof in the chain against the public ordered log $((D_1, C_{B_1}), \ldots, (D_E, C_{B_E}))$ — each entry pairing the epoch descriptor D_i with the binding epoch commitment C_{B_i} — and confirms that adjacent proofs agree on their boundary commitment: the output state commitment of π_i must equal the input state commitment of π_{i+1}. Provided the chain is intact, in log order, every π_i verifies, and the descriptors $(D_1, \ldots, D_E)$ tile the upload-log prefix without gaps or overlaps, the verifier learns that the final committed state is the result of applying all committed epochs in order.

The cost of this scheme falls on the verifier. After E epochs have closed, the verifier has to check E proofs and verify $E{-}1$ boundary equalities between consecutive proofs. Verification time scales as $\Theta(E)$ in the number of epochs. For sketches maintained over long data streams, the proof chain grows with the stream length, eventually exceeding what a bounded verifier can reasonably ingest and verify end-to-end.

A natural attempt to escape this bottleneck is to enlarge each epoch so that fewer transition proofs are produced. This only moves the cost: a very large epoch creates a large witness and circuit, increasing prover memory and proof-generation latency beyond the deployment's online budget, even though the verifier side of a

SNARK is largely succinct. The framework therefore needs both local transition proofs of bounded size and a history-composition layer that prevents the verifier from checking all past transitions; Section 4.2 collapses the chain into a single root proof via recursive composition.

4.2 Collapsing the chain via recursive composition

The per-epoch chain of §4.1 certifies each transition locally, but it never lets one proof speak for another. Boundary-state commitments tie each π_i to its neighbours structurally, yet no proof in the chain ever *verifies* any of the others, so checking one proof tells the verifier nothing about the rest; the verifier ends up paying for all E proofs. What we need is a primitive that lets one root proof subsume all earlier transition proofs, so the verifier checks one object regardless of how long the stream is. (Throughout this subsection we write b_i as shorthand for one closed epoch B_i, and C_{b_i} for its commitment; the item-wise case is $|B_i| = 1$.)

The primitive is recursive proof composition [15, 30]. A zero-knowledge proof system (§2.1) reduces a computation to an arithmetic circuit and emits a certificate that the circuit accepted its inputs; the system's *verifier* is itself a deterministic algorithm, and nothing stops us from encoding that verifier as a circuit and asking a fresh proof to certify a computation that *contains* the verifier as a subroutine. In the chain variant of recursive composition, a recursive circuit verifies the previous accumulator proof and incorporates the new local transition proof, emitting a single new proof that certifies both at once. In the tree and forest variants developed in §4.3, a merge circuit verifies one or two child interval proofs, checks that their public boundaries compose, and emits a parent interval proof. In both cases the new proof is a transitive certificate for the history certified by its children, and the database analogue is an incrementally maintained materialised view whose provenance certificate retroactively verifies the previous certificate inside the same circuit, so a single root certificate stands for the entire maintenance history. Applied to the per-epoch chain of §4.1, the prover emits a single $\Pi^{(E)}$ certifying that all E transitions were sound, in order, against the ordered-epoch digest $H_{[1,E]}$ computed from the public records $(D_1, C_{b_1}), \ldots, (D_E, C_{b_E})$. The rest of the subsection makes this concrete: we fix the static object that recursive composition produces, then the local rule that builds it from smaller pieces, then how the prover delivers it to the verifier.

Interval proofs and the merge atom. For an interval $[i, j]$ of epochs, an *interval proof* $\Pi_{[i,j]}$ certifies that the committed state at the end of the interval is reachable from the committed state before it:

$$\mathsf{Com}(S_{i-1}) \xrightarrow{b_i, b_{i+1}, \ldots, b_j} \mathsf{Com}(S_j).$$

Its public boundary is $\mathsf{pub}_{[i,j]} = (i,\ j,\ \mathsf{Com}(S_{i-1}),\ \mathsf{Com}(S_j),\ H_{[i,j]},\ \mathsf{M})$, where $H_{[i,j]}$ is a collision-resistant digest of the ordered epoch records $(D_i, C_{b_i}), \ldots, (D_j, C_{b_j})$ that binds both membership and order, including the descriptors that pin each epoch's upload-log slice — omitting, replacing, or reordering an epoch (or substituting its descriptor) changes the public input. A leaf interval is a single epoch with $H_{[j,j]} = \mathsf{H}(j,\ D_j,\ C_{b_j})$. Because the transition proof π_j has public input $(D_j, \mathsf{Com}(S_{j-1}), C_{B_j}, \mathsf{Com}(S_j), \mathsf{M})$ but the interval-proof public boundary is $(j, j, \mathsf{Com}(S_{j-1}), \mathsf{Com}(S_j), H_{[j,j]}, \mathsf{M})$, we view $\Pi_{[j,j]}$ as either (i) the transition proof π_j presented together with its public epoch record (D_j, C_{B_j}), against which the verifier recomputes $H_{[j,j]}$, or (ii) a one-step wrapper proof that verifies π_j in-circuit and exposes only the interval-shaped boundary above. Either lifting suffices for the rest of the construction; we use whichever matches the chosen recursive backend.

The recursive operation that builds larger interval proofs from smaller ones is the *merge atom* (Figure 2). Given two adjacent interval proofs $\Pi_{[i,m]}$ and $\Pi_{[m+1,j]}$, the merge circuit verifies both children in-circuit, enforces that the shared boundary commitment $\mathsf{Com}(S_m)$ exposed as the left child's output matches the one exposed as the right child's input, combines the two ordered-epoch digests via $H_{[i,j]} = \mathsf{H}(i,\ m,\ j,\ H_{[i,m]},\ H_{[m+1,j]})$, and emits a single parent proof $\Pi_{[i,j]}$ on the outer boundary $(\mathsf{Com}(S_{i-1}), \mathsf{Com}(S_j))$. The intermediate $\mathsf{Com}(S_m)$ is absorbed — it no longer appears on the parent's public boundary — so a verifier holding $\Pi_{[i,j]}$ checks one ZKP and learns that both children were sound *and* agreed on their join point. Iterating the atom collapses an arbitrary-length committed history into a single root proof on demand. Concretely, suppose the first two epochs b_1, b_2 have been processed, with per-epoch proofs π_1, π_2 binding $\mathsf{Com}(S_0) \xrightarrow{b_1} \mathsf{Com}(S_1)$ and $\mathsf{Com}(S_1) \xrightarrow{b_2} \mathsf{Com}(S_2)$; lifting each π_j to a leaf interval gives $\Pi_{[1,1]}, \Pi_{[2,2]}$, and one application of the merge atom verifies both leaves, checks that $\mathsf{Com}(S_1)$ agrees, combines digests as $H_{[1,2]} = \mathsf{H}(1, 1, 2, H_{[1,1]}, H_{[2,2]})$, and emits $\Pi_{[1,2]}$ with public boundary $(1,\ 2,\ \mathsf{Com}(S_0),\ \mathsf{Com}$ The intermediate $\mathsf{Com}(S_1)$ has been absorbed; the parent exposes only the outer boundary. We continue this example in §4.3 as more epochs arrive.

The chain as the simplest schedule, and the root interface. The simplest way to apply the merge atom to a streaming history is to maintain one accumulating root proof $\Pi^{(t)}$, extending it by one merge each time a fresh leaf arrives: $\Pi^{(1)} = \pi_1$, and at $t \geq 2$ the prover merges $\Pi^{(t-1)}$ with $\Pi_{[t,t]} = \pi_t$ to obtain $\Pi^{(t)}$ (Figure 3a). The prover therefore always holds a current root proof; a query arriving at any moment reads $\Pi^{(t)}$ directly, so the query latency is $O(1)$ *by construction.* The online cost per epoch is one merge.

Folding-style realisations [30] keep that per-epoch cost constant in practice: instead of running the full in-circuit ZKP verifier at every step, the prover folds the new instance into an accumulator at constant cost. In a folding realisation the online object may be an accumulator rather than a finalised SNARK; when a verifier or query requires a conventional succinct proof, the prover wraps the accumulator into one. Thus the chain delivers a current accumulator (or finalised root) at every step, but final proof materialisation may incur backend-dependent latency.

For a stream prefix of length t, the prover therefore publishes the pair

$$\big(R^{(t)},\ \Pi^{(t)}\big), \qquad R^{(t)} = \mathsf{H}\big(1,\ t,\ \mathsf{Com}(S_0),\ \mathsf{Com}(S_t),\ H_{[1,t]},\ \mathsf{M}\big),$$

where $\Pi^{(t)} = \Pi_{[1,t]}$ is the root interval proof. Both objects are needed: $R^{(t)}$ alone is a $O(1)$-size digest, not a proof. To tie the root all the way back to the public log, the verifier recomputes $H_{[1,t]}$ from the public ordered epoch records $((D_1, C_{B_1}), \ldots, (D_t, C_{B_t}))$, recomputes $R^{(t)}$ from the resulting $\mathsf{pub}_{[1,t]}$ exposed by $\Pi^{(t)}$, and then verifies $\Pi^{(t)}$ against $\mathsf{pub}_{[1,t]}$. Equality of the recomputed $H_{[1,t]}$

Figure placeholder. One recursive merge step (the atom of the construction). *Inputs (bottom of the figure):* two child interval proofs. The left child $\Pi_{\text{left}} = \Pi_{[i,m]}$ carries the public boundary $(\mathsf{Com}(S_{i-1}), \mathsf{Com}(S_m), H_{[i,m]})$; the right child $\Pi_{\text{right}} = \Pi_{[m+1,j]}$ carries $(\mathsf{Com}(S_m), \mathsf{Com}(S_j), H_{[m+1,j]})$. Draw each child as a small box with three labelled tabs on its public boundary, and use blue for state-commitment boundaries that will survive into the parent, grey for the shared $\mathsf{Com}(S_m)$ that will be absorbed, orange for the ordered update digests H. *Centre of the figure:* the recursive merge circuit, drawn as a larger box with a small ZKP-verifier icon inside. Internally the box performs three operations: (i) it runs the ZKP verifier on each child against its own public boundary; (ii) it checks that the shared $\mathsf{Com}(S_m)$ exposed as the left child's *output* equals the one exposed as the right child's *input*, highlighted with a dashed equality bracket between the two grey tabs; (iii) it combines the two ordered update digests via $H_{[i,j]} = \mathsf{H}(i, m, j, H_{[i,m]}, H_{[m+1,j]})$, shown as an orange merge node. *Output (top of the figure):* one parent interval proof $\Pi_{\text{parent}} = \Pi_{[i,j]}$ exposing only the outer boundary $(\mathsf{Com}(S_{i-1}), \mathsf{Com}(S_j), H_{[i,j]})$; the shared $\mathsf{Com}(S_m)$ does not appear on the parent's public boundary (it has been "hidden" behind the merge). *Caption hook:* include a small inset diagram on the right suggesting that iterating this step over a forest of size-2^k subtrees yields a single root proof on demand (forward-pointer to Figure 4).

Figure 2: The atom of recursive composition. Two child interval proofs go in, one parent interval proof comes out. The merge circuit runs the ZKP verifier on each child inside a new circuit, enforces that the shared boundary state commitment $\mathsf{Com}(S_m)$ agrees on both sides, and combines the ordered update digests. Iterating this step over a forest of size-2^k subtrees (Figure 4) collapses an arbitrary-length committed history into one root proof checkable in $O(1)$ time.

with the digest the root proof exposes is what binds the succinct root to the actual public log; one succinct proof check suffices, regardless of t. The chain has one structural limitation, however: each merge depends on the previous one finishing, so the schedule is purely sequential. A multi-core or GPU prover has no way to use its parallel resources across the stream, and on high-velocity streams this becomes the bottleneck. How the prover can *parallelise* the merge work to keep up with high-velocity streams, trading some of the chain's $O(1)$ query latency for parallel proving, is the subject of §4.3.

4.3 Scheduling recursive composition over a streaming history

The chain of §4.2 pays exactly one merge per epoch on the critical path, so a query at any time receives a ready root proof in $O(1)$ verifier work — provided the prover has caught up. Let v denote the epoch arrival rate (closed epochs per second), τ the wall-clock cost of one merge proof under a fixed backend thread budget, and k the number of such merge jobs that can run concurrently. (A single merge proof may itself be multi-threaded; τ absorbs that thread budget and k counts merge-level parallelism on top.) A sequential chain processes $1/\tau$ merges per second, so when only $k = 1$ merge worker is available the chain falls behind by $v - 1/\tau$ merges per second whenever $v\tau > 1$. Once a backlog B has accumulated, a query that should observe the latest version pays a catch-up cost of $B\tau$ rather than $O(1)$. Concretely, high-velocity streams — financial tick data, blockchain analytics, IoT telemetry — routinely produce 10^3–10^4 records per second; in the item-wise or small-epoch regime where each closed epoch carries a single record or a sub-millisecond mini-batch, this translates to $v = 10^3$ closed epochs per second. A single core paying $\tau = 10$ ms per merge clears only 100 epochs per second and falls behind by 900 per second, so one second of streaming leaves a 9 s catch-up debt for the next query. At the other extreme of large epochs (e.g. one-second sealing windows on the same record stream), v drops to a few epochs per second and the chain comfortably keeps up; the velocity question sharpens as epochs shrink. The chain's $O(1)$ query latency is therefore conditional, not unconditional; the rest of this subsection asks how to schedule the merges so disjoint pieces of the history are proved concurrently. Two structural refinements answer that question and differ in how they treat a fixed window versus a live stream.

From a chain to parallel trees. When the stream length E is known in advance and a power of two, the natural parallelisation organises the E leaf proofs as a balanced binary tree of depth $\lceil \log_2 E \rceil$ (Figure 3b). Every merge in the tree applies the same fixed-shape circuit; disjoint subtrees can be built in parallel across cores; the verifier still checks one root proof. The trade-off versus the chain is concrete: appending a new leaf regenerates the $\lceil \log_2 E \rceil$ merge proofs along the spine from the new leaf to the root, so the per-epoch latency to refresh the root is $\Theta(\log E)$ rather than $O(1)$. On k cores the same-level merges at each depth run concurrently, clearing the $v\tau > 1$ regime as long as k scales with v. Query latency is higher than the chain's when the prover must refresh the root on demand, but throughput on a multi-core prover is far higher; the balanced binary tree is therefore the natural choice for windowed batch processing where the prover commits one tree per window. Live streams break the windowed assumption, however: E is unknown and queries arrive at unpredictable moments. We need a structure that accommodates any current length t, admits cheap incremental extension as new leaves arrive, and can deliver a single root proof at any moment on demand.

Streaming forests and the cascade rule. For a live stream we organise the current history $[1, t]$ as a forest of perfect interval-proof subtrees whose sizes are the powers of two appearing in the binary representation of t. Writing $t = 2^{k_1} + 2^{k_2} + \cdots + 2^{k_\ell}$ with $k_1 > k_2 > \cdots > k_\ell \geq 0$, the forest holds $\ell \leq \lceil \log_2(t+1) \rceil$ surviving subtree roots, one of size 2^{k_r} for each set bit; at $t = 11 = 8 + 2 + 1$ the forest holds three roots $\Pi_{[1,8]}, \Pi_{[9,10]}, \Pi_{[11,11]}$ (Figure 3c). The skeleton is the Bentley-Saxe static-to-dynamic transformation [4], and the layout is structurally analogous to a Merkle Mountain Range [33]; we borrow only the organisation that schedules the

Figure placeholder. Three schedules for composing interval proofs over a streaming history. (a) *Linear chain*: a single accumulating interval $\Pi_{[1,t]}$ is extended by one merge per arrival; recursion depth of the root proof grows linearly with t. (b) *Balanced binary tree* (idealised, $E = 8$): leaves $\Pi_{[1,1]}, \ldots, \Pi_{[8,8]}$ at the bottom, internal merges at intervals $[1, 2], [3, 4], [5, 6], [7, 8], [1, 4], [5, 8], [1, 8]$; requires E known in advance and a power of two. (c) *Streaming forest* (general t, illustrated at $t = 11 = 8 + 2 + 1$): three surviving subtree roots $\Pi_{[1,8]}, \Pi_{[9,10]}, \Pi_{[11,11]}$ of sizes 8, 2, 1. Draw the on-demand chaining as dashed merges from left to right that produce a single root $\Pi^{(11)}$ when a query or verifier request triggers finalisation. All three publish the same verifier interface $(R^{(t)}, \Pi^{(t)})$.

Figure 3: Three composition schedules for interval proofs. Panel (a) is the chain of §4.2 reproduced for comparison; panels (b) and (c) are the parallel schedules developed in this section. The chain is sequential with $O(1)$ query latency; the balanced binary tree is parallelisable but assumes a fixed, power-of-two history; the streaming forest accommodates any t and finalises a single root proof on demand.

recursive merges, and do not derive membership proofs from the forest.

When π_{t+1} arrives, the prover inserts a fresh size-1 leaf $\Pi_{[t+1,t+1]}$ on the right. If a size-1 subtree was already the rightmost surviving root, the two merge into a size-2 subtree via one application of the merge atom; if a size-2 subtree was also present, the cascade continues, and so on until no two equal-sized neighbours remain. The cascade visits at most $\lceil \log_2(t+1) \rceil$ levels and emits $O(\log t)$ merges in the worst case, with $O(1)$ amortised over the stream by the standard binary-counter argument; at any moment the prover can compose the surviving forest roots into a single root proof $\Pi^{(t)}$ by chaining them left-to-right with $\ell - 1 < \lceil \log_2(t+1) \rceil$ additional merges, and publish $(R^{(t)}, \Pi^{(t)})$ to the verifier.

It is worth being precise about where the parallelism actually lives, because a single cascade is itself sequential: at $t = 8$, with prior forest $\{\Pi_{[1,4]}, \Pi_{[5,6]}, \Pi_{[7,7]}\}$, the cascade fires three dependent merges

$$M_1 : \ \Pi_{[7,7]} + \Pi_{[8,8]} \rightarrow \Pi_{[7,8]},$$
$$M_2 : \ \Pi_{[5,6]} + \Pi_{[7,8]} \rightarrow \Pi_{[5,8]},$$
$$M_3 : \ \Pi_{[1,4]} + \Pi_{[5,8]} \rightarrow \Pi_{[1,8]},$$

in which M_{j+1} consumes M_j's output and cannot start earlier. The parallelism the forest unlocks lives elsewhere: the surviving roots are produced by independent subtree histories whose internal merges occupy disjoint intervals ($\Pi_{[1,4]}$ and $\Pi_{[5,6]}$ above were built over earlier inserts on disjoint ranges), so subtree-internal merges run concurrently across cores as they arise; the $\ell - 1$ chaining merges that produce $\Pi^{(t)}$ on demand are left-to-right dependent but their *inputs* are already-completed surviving roots whose preparation is parallelisable; and cascades from different inserts can overlap when their target subtrees are disjoint. The on-demand cost of delivering a single root proof is therefore $O(\log t)$ — one logarithmic factor higher than the chain's $O(1)$ ready root, exchanged for the across-insert parallelism the chain could not provide. How the prover orchestrates this work over time — whether the chain is materialised eagerly, lazily, or somewhere in between — is a separate question, treated next in §4.4.

4.4 Maintenance policies: lazy, eager, smoothed

The streaming forest of §4.3 fixes the static structure of recursive composition; what remains is to choose *when* the prover folds the surviving roots into the $\Pi^{(t)}$ that a verifier reads. Three peak-chain policies cover the common cases; Table 1 summarises their cost profiles, and the rest of this subsection unpacks how each policy distributes the same amortised $O(1)$ work per insert across insert time and query time.

The simplest is the *lazy* policy: the prover does only the cascade itself on each insert ($O(1)$ amortised, $O(\log t)$ worst case at $t = 2^k$), and a query when the forest is stable pays $\ell - 1 \leq \log_2(t+1)$ chaining merges to assemble $\Pi^{(t)}$. A query mid-cascade first finishes the in-progress cascade and then performs the chaining. Inserts are as cheap as they can be, but queries pay the full finalisation on demand — appropriate when queries are rare relative to inserts.

The *eager* policy front-loads the chaining: after every insert the prover incrementally repairs the maintained peak chain over the new surviving roots (one extra merge per peak consumed by the cascade, no full rebuild). Inserts pay $O(1)$ amortised with $O(\log t)$ cascade-aligned bursts at $t = 2^k$ (when the cascade and the peak-chain repair coincide). Under an *atomic-insert* convention — a query is admitted only after the current insert has committed — the eager policy presents a ready root and queries pay 0, even immediately after a cascade. Without atomicity, a query that arrives during an in-flight cascade burst either waits for the burst to complete or is answered against the previous closed version, depending on the release policy of §3.3.

The eager policy's burst at $t = 2^k$ is the largest spike in insert latency. The *smoothed* policy flattens that burst by holding the deferred root-maintenance work in a pending dependency-respecting queue Q_t and draining at most c of its entries per insert: when a cascade fires, the prover enqueues its merges into Q_t, and the current insert and each subsequent insert drain up to c ready merges from Q_t in dependency order. Q_t contains exactly the root-maintenance work needed to restore the eager state: the delayed cascade merges, followed by any peak-chain repair merges required to expose the canonical root $\Pi^{(t)}$ for the latest version. When $Q_t = \emptyset$, the maintained root is current by invariant. During smoothing the prover maintains a *relaxed forest* together with Q_t as a dependency-respecting merge DAG: the usual binary-forest invariant (no two equal-size adjacent surviving roots) is restored only after Q_t drains; until then, new leaves are appended to the right and any newly enabled cascade merges are inserted into Q_t behind their predecessors. Each insert performs at most c root-maintenance merges: the ordinary peak-chain repair when the forest root changes, plus queued backlog work when Q_t is non-empty. The smoothed policy therefore adds at most $c\tau$ of merge work per insert rather than spiking to $O(\log t)\tau$ at cascade alignment. A query while $Q_t = \emptyset$ pays 0. A query while Q_t is non-empty

Table 1: Cost profile of *root-maintenance merge work* under the three peak-chain policies and the query-burst refinement of the lazy policy. The table excludes the per-epoch transition proof itself (which every policy pays equally and is the subject of §4.1); it shows only the recursive-composition work used to maintain or finalise $\Pi^{(t)}$. On the insert path, all policies amortise to $O(1)$ recursive-maintenance work per epoch; query-triggered finalisation is accounted for separately in the query columns and may be paid repeatedly under heavy query loads. In the mid-cascade column, B denotes the standing backlog and ℓ the surviving-peak count.

Policy	Steady insert	Cascade insert	Query (current)	Query (mid-cascade)
Lazy	0	$O(\log t)\,\tau$	$O(\log t)\,\tau$	$O(\log t)\,\tau$
+ query-burst	0	$O(\log t)\,\tau$	$O(\log t)\,\tau$ on $k \geq \ell/2$ workers	finish cascade + $O(\log \ell)\,\tau$
Eager	τ	$O(\log t)\,\tau$	0	0 (atomic) or wait
Smoothed c=1	$\leq\tau$ per epoch (τ for peak repair; $\leq\tau$ while draining backlog)	0	drain Q_t: $B\tau$ immediate, $O(B/c)$ ins	

has two choices: if it can wait for background draining, the root becomes current after at most $\lceil |Q_t|/c \rceil$ further insert steps — the *background lag*, bounded by the standing queue depth and read off Q_t directly; if it must answer immediately for the latest version, it pays the wall-clock cost of draining the dependency-respecting queue, which for a single binary-counter cascade is one dependent carry chain of $|Q_t|\tau$, bounded by $O(\log t)\tau$ in the worst transient case. Setting $c = \infty$ recovers the eager policy (immediate drain at the firing insert); $c = 1$ matches the long-run average merge rate and gives the flattest insert curve at the price of a transient catch-up cost on a query that arrives mid-cascade.

The three policies share $O(1)$ amortised total work per insert; they differ only in how the work is distributed: lazy concentrates the chaining at query time, eager front-loads it into the cascade insert, smoothed amortises the cascade across several inserts. The lazy/eager dichotomy is the standard decomposable-data-structure trade-off, formally analysed in the competitive sense by Mathieu et al. [23].

Query-burst finalisation as an optional optimisation. The lazy policy's $O(\log t)$ on-demand cost chains ℓ surviving roots left-to-right. Because those roots come from disjoint subtree histories, they are mutually independent; their chaining can therefore be reshaped as a balanced peak-folding tree of depth $\lceil \log_2 \ell \rceil$ over the ℓ peaks (the schedule of Figure 3b applied to the surviving peaks only), when this schedule is the one declared in M, and its tree root is emitted as $\Pi^{(t)}$. On k cores the wall-clock cost is the work/span bound

$$\sum_{j=0}^{\lceil \log_2 \ell \rceil - 1} \left\lceil \frac{\ell / 2^{j+1}}{k} \right\rceil \tau,$$

which collapses to $\lceil \log_2 \ell \rceil \tau$ once $k \geq \ell/2$ and degrades gracefully as k shrinks. The optimisation is well-defined only because the surviving roots are mutually independent; it does *not* apply to the smoothed policy's pending queue Q_t when the queue holds a single cascade's dependent carry chain — there M_{j+1} consumes M_j's output, so the entries cannot be reordered into a balanced shape and must be drained sequentially. Query-burst is therefore an off-by-default refinement of the lazy policy (or the smoothed policy applied to the independent surviving peaks, separately from the queue drain); it cuts the on-demand chaining cost when many surviving peaks have accumulated, without changing any cost in steady state or under eager.

Because the interval digest of §4.2 records its split point, different finalisation schedules produce different bracketings and therefore different digest trees over the same ordered interval $[1, t]$. A left-to-right chained root and a balanced-tree root over the same ℓ peaks both certify the same committed history but expose different internal shapes. To avoid presenting an auditor with multiple valid roots for the same version, we adopt a *canonical-per-deployment* rule: each deployment fixes one finalisation schedule (chain or balanced) in the public metadata M, so $R^{(t)}$ is unique for that deployment and version, and the verifier checks the delivered proof against the declared schedule. Under this rule, query-burst is not an ad-hoc change to the root shape at query time; it is the lazy finalisation procedure for deployments whose metadata declares the balanced peak-folding schedule, while deployments that declare the chain-shaped schedule disable query-burst and pay the sequential chaining cost.

Choosing among the policies. Which policy to use depends on the epoch velocity v, the per-merge cost τ, the query rate q, and the available worker count k. At low velocity ($v\tau \ll 1$) the chain of §4.2 keeps up on a single core and no parallelism is needed. At high velocity ($v\tau > 1$) with rare queries, the lazy forest amortises inserts to $O(1)$ and exploits parallelism across subtrees; rare queries pay an $O(\log t)$ finalisation. At high velocity with moderate query rates, the eager peak chain pays the cascade-aligned $O(\log t)$ burst at insert time so queries observe a ready root. At high velocity with bursty queries, the smoothed peak chain with c=1 keeps inserts uniform and the query-burst optimisation folds the independent surviving peaks in $O(\log \ell)$ depth on k workers; a query that lands mid-cascade still pays $|Q_t|\tau$ to drain the dependent carry chain, but the chaining over the surviving peaks runs in parallel rather than sequentially. This is zkSketch's default candidate configuration for high-velocity workloads with intermittent queries; the evaluation of §6 measures whether the steady-state insert smoothing is worth the mid-cascade query lag for a given deployment.

Example 4.1 (Three cascade events under the four policies)**.** We trace the policies through two cascade events on the stream of Figure 4; the trivial $t = 2$ depth-1 cascade is omitted (all four policies do the same one-merge work). At $t = 4$ (cascade of depth 2) the forest goes from $\{\Pi_{[1,2]}, \Pi_{[3,3]}\}$ to $\{\Pi_{[1,4]}\}$ via $M_1 = \Pi_{[3,3]} + \Pi_{[4,4]} \to \Pi_{[3,4]}$ then $M_2 = \Pi_{[1,2]} + \Pi_{[3,4]} \to \Pi_{[1,4]}$. Lazy and eager both pay 2τ at $t = 4$; smoothed (c=1) enqueues $\{M_1, M_2\}$ into Q_4, drains M_1 at $t = 4$, and drains M_2 at $t = 5$. If epoch 5 has closed and the policy needs the canonical root for version 5, a further peak-chain repair merge $\Pi_{[1,4]} + \Pi_{[5,5]} \to \Pi^{(5)}$ is enqueued behind M_2, so Q_5 stays non-empty until that repair drains too. At $t = 8$ (cascade of depth 3) the forest goes from $\{\Pi_{[1,4]}, \Pi_{[5,6]}, \Pi_{[7,7]}\}$ to $\{\Pi_{[1,8]}\}$ via the three dependent merges M_1, M_2, M_3 defined in §4.3. Lazy and eager pay 3τ at $t = 8$; smoothed (c=1) enqueues $\{M_1, M_2, M_3\}$ into Q_8 and drains them at $t = 8, 9, 10$ because M_2 depends on M_1 and M_3 on M_2. A query at $t = 9$ under smoothed observes $Q_9 = [M_3, M_4]$ — the remaining cascade tail plus the peak-chain

Figure placeholder. Streaming forest after $t = 1, 2, 3, 4$ committed epochs. *Panel t=1:* the forest holds a single size-1 leaf $\Pi_{[1,1]}$. *Panel t=2:* the new leaf $\Pi_{[2,2]}$ cascades with the existing size-1 root; the forest collapses to one size-2 subtree $\Pi_{[1,2]}$, with the absorbed $\mathsf{Com}(S_1)$ shown crossed out behind the merge. *Panel t=3:* the new leaf $\Pi_{[3,3]}$ is size-1 and meets a size-2 neighbour; no cascade fires. Two surviving roots $\Pi_{[1,2]}, \Pi_{[3,3]}$. *Panel t=4:* the new leaf $\Pi_{[4,4]}$ cascades twice: first with $\Pi_{[3,3]}$ into $\Pi_{[3,4]}$, then with $\Pi_{[1,2]}$ into $\Pi_{[1,4]}$. The forest collapses to a single size-4 subtree. Dashed arrows above each panel indicate the on-demand finalisation that chains surviving roots into one $\Pi^{(t)}$. Use orange for ordered-digest combinations and grey for absorbed boundary commitments.

Figure 4: Streaming forest evolving with the running example (Example 4.1). Each panel shows the forest after one more epoch closes; cascades fire whenever adjacent subtrees have equal size. At any time, on-demand finalisation (dashed) produces a single root proof $\Pi^{(t)}$ verifiable in $O(1)$ time. Under the eager peak-chain policy of §4.4, the dashed chain in each panel is what the prover maintains continuously, with the cascade at t=4 repairing the maintained peak chain over the new single peak.

repair merge $M_4 : \Pi_{[1,8]}+\Pi_{[9,9]} \rightarrow \Pi^{(9)}$ enqueued behind M_3 to expose the canonical root for the new leaf — and pays a sequential drain of 2τ before $\Pi^{(9)}$ is current. The dependency chain inside Q_t cannot be reordered into a balanced shape (M_4 consumes M_3's output), so query-burst does not help in this case. Where it *does* help is the lazy policy when many independent surviving peaks have accumulated: at $v = 10^3$ epochs per second and a stream of length $t \approx 10^6$ (about 17 minutes of streaming) just before the next major cascade, the surviving-peak count is $\ell = \lfloor \log_2 t \rfloor \approx 20$, sequential chaining pays $(\ell - 1)\tau \approx 190$ ms, and the declared balanced peak-folding tree pays $\big(\lceil 10/k \rceil + \lceil 5/k \rceil + \lceil 3/k \rceil + \lceil 2/k \rceil + \lceil 1/k \rceil\big)\tau$ wall-clock time — about 60 ms on $k = 8$ merge workers (a $\sim 3\times$ improvement) and approaching $\lceil \log_2 \ell \rceil \tau \approx 50$ ms once $k \geq \ell/2 = 10$. Query-burst does *not* beat a caught-up chain — whose root is already current and asymptotically optimal at $O(1)$ — but it beats the catch-up cost a lazy forest would otherwise pay on a query that arrives with many peaks accumulated.

4.5 Answering queries during streaming

Queries arrive interleaved with updates and must observe a clean prefix of the stream. The streaming forest of Section 4.3 supports this directly.

When a query arrives at logical time j, the prover first closes the current open epoch if any uploads have arrived since the last committed version: let h be the last closed version; the open batch is sealed as the next epoch B_{h+1}, the prover emits $\mathsf{Com}(S_{h+1})$ and the corresponding per-epoch transition proof π_{h+1}, and the new leaf is inserted into the streaming forest via the cascade of Section 4.3. Let $t = \mathrm{ver}(j)$ be the resulting closed sketch version, in the notation of §3.1.

The prover then invokes the finalisation procedure of the active peak-chain policy (§4.4). Under lazy, it chains the surviving forest roots into a single root proof $\Pi^{(t)}$ on demand (optionally via query-burst when the deployment declares the balanced schedule and many peaks have accumulated). Under eager, it reads the maintained root proof directly. Under smoothed, the default query-prefix semantics of §3.1 require an answer bound to $t = \mathrm{ver}(j)$, so the prover first drains the pending dependency chain in Q_t needed to finalise version t and then returns $(\Pi^{(t)}, R^{(t)})$. A deployment may additionally expose an explicit *as-of* read mode in which a query is labelled as targeting an earlier finalised version $t' < t$; such an answer is bound to $(\Pi^{(t')}, R^{(t')})$ and does *not* claim to answer the logical-time query at version $\mathrm{ver}(j)$, so it does not satisfy query-prefix soundness (Property 3 of §3.5) for the time-j query. In every case the delivered root pair certifies that the bound committed state is reachable from $\mathsf{Com}(S_0)$ by applying the corresponding committed epochs in order.

For the query itself, let s denote the version actually bound by the delivered root: $s = t$ under the default query-prefix semantics, and $s = t'$ under an explicit as-of read. When the declared release policy *policy* is the identity, the decoded value *dec* is public and the prover emits it together with a *decode proof* π_q certifying

$$dec = A_\tau.\mathsf{Decode}_q(S_s),$$

where the public input of π_q binds to the $\mathsf{Com}(S_s)$ that appears on the public boundary of $\Pi^{(s)}$; here $ans = dec$. When *policy* is non-trivial (suppression, rounding, noise addition from a committed seed ρ), publishing *dec* would defeat the policy. In that mode *dec* remains a private witness to a combined decode-and-release proof π_q that certifies both $dec = A_\tau.\mathsf{Decode}_q(S_s)$ and $ans = policy(dec;\ \rho)$ against the public boundary of $\Pi^{(s)}$ and the committed seed; only the released *ans* is published. The tuple $(\Pi^{(s)}, \pi_q, ans)$ therefore ties the published answer to exactly the prefix of committed epochs through version s and to the declared release pipeline; under default semantics $s = \mathrm{ver}(j)$ and this is precisely the query-prefix soundness guarantee of Property 3 of §3.5.

5 Circuit Designs for Sketches

§4 fixed the proof-orchestration layer: per-epoch transition proofs as the basic proving unit, recursive history composition for $O(1)$-cost verification. It deferred the question this section answers: *how is one per-epoch transition proof constructed?* That is, given a committed epoch B_t, an old committed state $\mathsf{Com}(S_{t-1})$, and the published new state $\mathsf{Com}(S_t)$, what does the circuit certify and how? We build the answer step by step.

The strawman: replay the plaintext update inside the circuit. The most direct realisation is to encode the plaintext sketch update record-by-record. We use hatted intermediate states $\widehat{S}_0, \widehat{S}_1, \ldots, \widehat{S}_m$ for the per-record local trajectory inside one epoch to keep them distinct from the global per-epoch states S_{t-1}, S_t of the framework, with $\widehat{S}_0 = S_{t-1}$ and $\widehat{S}_m = S_t$. For each local row $p \in [m]$ (with the row's record $u_p \in B_t$) the circuit (a) opens u_p from C_{B_t}, (b) evaluates the keyed hashes $\{h_g(u_p)\}_{g \in [d]}$ in-circuit, (c) opens the touched cells of the current intermediate state $\widehat{S}_{p-1}$ from a commitment $\mathsf{Com}(\widehat{S}_{p-1})$, (d) updates those cells per the sketch's algorithm to

produce $\widehat{S}_p$, and (e) rebinds the state commitment to $\mathsf{Com}(\widehat{S}_p)$. After $m = |B_t|$ such steps the final state $\widehat{S}_m$ is checked against the published $\mathsf{Com}(S_t)$.

Why the strawman is unaffordable. The strawman pays in three structural ways. First, the *witness must materialise all* $m+1$ *intermediate sketch states* $\widehat{S}_0, \widehat{S}_1, \ldots, \widehat{S}_m$, because each per-record step depends on the previous one. For a Count-Min state of $|S| = dw \approx 4.6 \times 10^5$ cells (the running parameters $d = 7$, $w = 2^{16}$) and a batch of $m = 1024$, this is $\sim 4.7 \times 10^8$ field elements in the witness for a *single* epoch proof — an order of magnitude beyond the largest practical ZK-SNARK circuit. Second, the m intermediate state commitments $\mathsf{Com}(\widehat{S}_0), \ldots, \mathsf{Com}(\widehat{S}_{m-1})$ must each be opened and rebound in-circuit, paying $\Theta(m)$ commitment-update work plus per-cell openings. Third, the keyed hash evaluations live inside the circuit per record, contributing $\Theta(mdH)$ constraints under any in-circuit hash family (Appendix B quantifies the contribution).

Why this is a different bottleneck from §4.1. §4.1 treated each epoch as a single proving unit and addressed the verifier-side cost of a long chain of per-epoch proofs. Each π_i was a black box there. The strawman above instead opens that black box: a single epoch is internally a group of m records that the prover has accumulated, and the strawman replays the plaintext sketch update record-by-record *inside* that epoch's circuit, paying its costs m times in the single proof's witness. §5 must therefore answer a question §4.1 did not address: *how are the m records that make up one epoch handled inside that epoch's circuit?* The strawman handles them sequentially, paying for m intermediate states; the rest of this section avoids that.

Our approach: certify the net effect, not the procedure. The observation that closes the gap is that the verifier does not care about the intermediate $\widehat{S}_1, \ldots, \widehat{S}_{m-1}$; the verifier only cares that the published $\mathsf{Com}(S_t)$ is the result of applying $A_\tau.\mathsf{UpdateBatch}(S_{t-1}, B_t)$. The circuit can therefore *certify the net effect* of B_t on the committed state without materialising any intermediate state: the prover supplies a compact *batch-effect certificate* describing the net effect of B_t on the committed sketch state, and the circuit verifies the certificate is consistent with C_{B_t}, $\mathsf{Com}(S_{t-1})$, and $\mathsf{Com}(S_t)$. The certificate's shape depends on the sketch family's pattern: for Pattern A linear sketches (Count-Min) it is a sparse *delta certificate* with witness size $\Theta(m + U_{\max}d + T_{\max})$ in the grouped realisation (or $\Theta(md + T_{\max})$ without key preaggregation), where $U_{\max}$ is the padded grouped-key capacity and $T_{\max}$ is the padded touched-counter capacity; for Pattern A max sketches (HyperLogLog) it is a sparse register-rebind certificate of the same asymptotic shape; for Pattern B sketches (SpaceSaving, GK) it is a bounded replacement certificate whose size is determined by the public summary capacity and the number of fusion/eviction rounds. In all cases the witness shrinks from $\Theta(m|S|)$ in the strawman to a quantity that depends only on the certificate shape, not on the intermediate-state chain.

The rest of §5 develops this idea. §5.1 states the abstract batch-effect maintenance relation $\mathcal{R}^{\tau}_{\mathsf{upd}}$ that every sketch family's per-epoch transition circuit must satisfy; §5.2 packages the per-family choices as a seven-slot interface Π_τ and shows the algebra of slot S4 forces every supported family into one of two structural patterns; §5.3 and §5.4 walk Pattern A (Count-Min) and Pattern B (Greenwald–Khanna) end-to-end at the gate level; §5.6 states coverage and limits. HyperLogLog and SpaceSaving follow the same template and appear in Appendices C–D.

5.1 Batch-Effect Certificates

The transition circuit for one epoch should not replay the plaintext sketch update record by record. The verifier does not need to see the intermediate states

$$S_1, S_2, \ldots, S_{m-1}$$

inside the epoch. It only needs to be convinced that the published state commitment C_{new} is obtained by applying the declared sketch update algorithm to the old committed state C_{old} and the committed epoch C_{B_t}. We therefore use a *batch-effect certificate*: a bounded witness describing the net effect of the epoch on the sketch state.

For additive sketches such as Count-Min, the batch effect is a sparse counter-update vector: a list of sketch counters and the net amount added to each counter. For non-additive sketches, the same principle gives a different certificate shape. For example, HyperLogLog certifies a register-wise maximum update, while structured summaries such as Greenwald–Khanna certify that the new bounded summary is a valid compressed summary of the old state plus the epoch. Thus, "batch-effect certificate" is the umbrella term; "delta" is reserved for the additive case where the effect is a sparse vector of numerical counter changes.

Abstract update relation. For a sketch family τ, the per-epoch transition circuit proves the relation

$$\mathcal{R}^{\tau}_{\mathsf{upd}} = \left\{ (x_t, w_t) : \begin{array}{r} \mathsf{Open}(C_{\mathsf{old}}, S_{t-1}) \ \wedge \ \mathsf{Open}(C_{B_t}, B_t) \ \wedge \\ \mathsf{Open}(C_{\mathsf{new}}, S_t) \ \wedge \ \mathsf{SliceOK}(D_t, B_t) \ \wedge \\ S_t = A_\tau.\mathsf{UpdateBatch}(S_{t-1}, B_t) \end{array} \right\},$$

where $\mathsf{Open}(C, X)$ is the commitment-opening predicate (with the opening carried in w_t) and $\mathsf{SliceOK}(D_t, B_t)$ asserts that the private rows of B_t open to exactly the upload-log slice declared by D_t. The relation is *semantic*: it states what committed objects must exist, not how the circuit witnesses them. Concrete circuits need not open the entire state — Pattern A circuits realise the state-opening predicates by sparse openings plus a commitment-level update proof (so untouched cells are bound without being opened), while Pattern B circuits use bounded whole-state openings against a small fixed-capacity table. The public input is

$$x_t = (D_t, C_{\mathsf{old}}, C_{B_t}, C_{\mathsf{new}}, \mathsf{M}),$$

D_t is the public epoch descriptor of §3.3 that pins the epoch's contiguous upload-log slice, and the witness w_t contains the private batch opening, the old and new sketch-state openings required by the commitment scheme, and the family-specific certificate of the batch effect. The metadata M fixes the sketch type, public parameters, hash profile, counter representation, commitment scheme, and circuit capacities.

The important point is that w_t is not a transcript of all intermediate sketch states inside the epoch. Instead, it is a compact certificate that the batch effect was computed correctly from the committed batch and applied consistently to the committed old state. In addition, the input-binding check verifies that the private batch rows opened from C_{B_t} correspond to exactly the upload commitments

in the contiguous slice declared by D_t, so the same descriptor that anchors the upload-log partition in Property 2 also anchors the per-epoch input on which the family-specific certificate is built.

Certificate structure. The certificate is organized as a sequence of bounded witness tables and consistency checks. The input table is opened from C_{B_t}. The target table records the sketch locations affected by the epoch. The aggregate table contains one row per affected target, together with the net contribution to that target. The state-opening tables then certify that the old and new committed states differ exactly by these aggregate contributions.

For Count-Min, the target table is produced in two stages. The circuit first groups the epoch by key, obtaining rows (k_u, V_u), where V_u is the total value of key k_u in the epoch. It then computes the d counter locations $(i, h_i(k_u))$ for each grouped key and groups these rows by counter identifier. The resulting aggregate table contains one row $(i, b, \Delta_{i,b})$ for each touched counter, where

$$\Delta_{i,b} = \sum_{u:\, h_i(k_u)=b} V_u.$$

The first grouping avoids hashing duplicate keys repeatedly; the second grouping combines hash collisions between distinct keys.

Generic certificate obligations. Although the concrete tables differ by sketch family, every batch-effect certificate must establish four facts.

(1) **Input binding.** The private batch rows used by the circuit open from the public commitment C_{B_t} *and* match the contiguous upload-log slice declared by the public epoch descriptor D_t (i.e. the $\mathsf{SliceOK}(D_t, B_t)$ predicate of the relation above). The prover cannot invent, drop, or modify records without breaking either the commitment opening or the upload-log slice match.
(2) **Effect derivation.** The proposed batch effect is derived from the committed rows under the declared sketch algorithm and metadata M. For Count-Min, this means that each distinct key is mapped to one counter per sketch row by the declared hash functions. For other sketches, this slot is instantiated by the corresponding register, summary, or rank-update rule.
(3) **Effect aggregation.** If several rows contribute to the same update target, the circuit verifies that they are combined correctly. For additive sketches this is a sum; for max-based sketches it is a maximum; for structured summaries it is a family-specific invariant check.
(4) **State rebinding.** The old and new state commitments are consistent with applying the certified effect. For Pattern A linear sketches, this can be a sparse additive patch. For Pattern A max sketches, this is a register-wise max patch. For Pattern B sketches, this is a whole-state replacement checked against the summary invariant.

Circuit realisation. All certificates are expressed using standard table checks: permutation or lookup arguments for multiset equality, lexicographic comparison gates for sortedness, local recurrences for segment aggregation, and Mux gates for the conditional update / copy constraints used in fusion and eviction steps. The public circuit shape depends only on M (sketch parameters, hash profile, padded capacities, fusion-round count), not on the private number of active keys, touched cells, or fusions — which is exactly the circuit-obliviousness invariant of Property 4 of §3.5.

Algorithm 1: Batch-effect update circuit, sketch-agnostic

Input: Public $D_t, C_{B_t}, C_{\text{old}}, C_{\text{new}}, \mathsf{M}$; private certificate tables W_τ
Output: Proof that C_{new} is obtained by applying epoch B_t to C_{old} under the declared sketch family A_τ

1 Verify that the private batch table opens from C_{B_t}, matches the upload-log slice declared by D_t, and uses canonical dummy rows for inactive entries;
2 Use the sketch-specific target-generation gates to derive update targets from the batch rows under metadata M;
3 Verify that all contributions to the same target are combined according to the sketch-specific aggregation operator;
4 Verify the old and new state openings required by the chosen commitment scheme;
5 Verify that applying the certified batch effect to the old committed state yields the new committed state C_{new};
6 Verify metadata compatibility and expose the transition proof as the leaf payload consumed by the history-composition layer of §4.3;

Table 2: The seven slots of a sketch transition circuit. A family τ plugs into the framework by filling each slot with a gate or table that the circuit can verify with low-degree constraints.

Slot	Role in the per-epoch transition circuit
S1. Input row format	How a private batch row encodes a record (e.g. $(a, k$
S2. Target generation	How each batch row derives the sketch locations it a
S3. Aggregation operator	How contributions to the same target combine (sum
S4. State patch form	Sparse operator patch (additive delta for CM, max-re
S5. Commitment update check	Sparse vector-commitment update / rebind for Patte
S6. Query decode circuit	The per-family decode relation Decode_q
S7. Public capacity / padding rule	Padded sizes and circuit-shape constants written int

The proof follows the standard extract-then-bind structure. Knowledge soundness extracts the private certificate tables. Commitment binding pins the batch opening and the state openings. The circuit constraints then force the extracted certificate to be a correct batch effect under the declared sketch algorithm, and force the new commitment to bind to the result of applying that effect. Multi-epoch history binding is handled separately by the recursive history-composition layer of §4.3.

5.2 Interface and Pattern Classification

The abstract relation above hides the per-family choices behind A_τ and the family-specific certificate. Before walking the two pattern representatives end-to-end, we name the seven slots each sketch family must fill to instantiate the framework (Table 2) and the two structural patterns that emerge from the algebra of one slot (Table 3).

The dichotomy in Table 3 arises from slot S4: when the per-target operator is associative-commutative and the state is a fixed-shape array (CM, HLL), the patch is a sparse operator patch over touched

Table 3: Sketch families and their structural patterns. Pattern A covers families whose state update is a per-target operator that composes commutatively (sum for linear, max for HLL), so the patch is a sparse operator patch over touched cells. Pattern B covers families whose update is a bounded-state replacement with a deterministic fusion / eviction rule.

Family	Pattern	Slot-4 patch form
Count-Min (§5.3)	A (linear)	sparse additive delta
HyperLogLog (Appendix C)	A (max)	sparse max-rebind patch
SpaceSaving (Appendix D)	B (top-s retention)	whole-state replacement
Greenwald–Khanna (§5.4)	B (rank-interval fusion)	whole-state replacement

cells (additive delta for CM, max-rebind for HLL); when the state is a bounded ordered summary with a deterministic merge/eviction rule (SS, GK), the patch is a whole-state replacement. The remaining six slots specialise within each pattern but share the same circuit skeleton. §5.3 walks Count-Min as Pattern A's representative; §5.4 walks Greenwald–Khanna as Pattern B's representative; HyperLogLog and SpaceSaving follow the same template and appear in Appendices C–D.

Why these two representatives. We present Count-Min and Greenwald–Khanna in the main body because they exercise the two qualitatively different circuit patterns in the framework, and each surfaces a distinct family of reusable circuit-design techniques. Count-Min represents fixed-array sketches whose batch effect is a sparse per-target operator patch; it exposes the reusable sort-group-hash-aggregate-rebind pipeline (verified sorting, multiset-equality grouping, hash-target generation once per distinct key rather than once per record, segment-aggregation over collisions, sparse patching, and commitment-level rebinding of untouched coordinates). Greenwald–Khanna represents bounded ordered summaries whose update is certified as a deterministic whole-state replacement; it exposes the reusable sorted-union, parity-alternating fusion, deterministic compaction, and ordered-summary invariant-checking pipeline. HyperLogLog and SpaceSaving instantiate the remaining variants of these same patterns — HLL is Pattern A with sum replaced by max (sparse max-rebind), and SpaceSaving is Pattern B with rank-interval fusion replaced by top-s retention via the threshold-residual certificate — and are deferred to the appendix because they reuse the Count-Min and GK techniques rather than introducing new ones.

5.3 Count-Min Circuit Design

Assume that each uploaded record is a key–value pair (k, v), where v may be positive or negative (a signed linear Count-Min update; see the disclaimer below). A Count-Min sketch stores a $d \times w$ array of counters. For one record (k, v), the plaintext update adds v to one counter in each row:

$$S[i, h_i(k)] \leftarrow S[i, h_i(k)] + v, \qquad i \in [d].$$

A point query for key q returns

$$\min_{i \in [d]} S[i, h_i(q)].$$

The special case $v = 1$ gives the usual frequency-estimation setting. When negative v is enabled, the circuit certifies the declared signed linear Count-Min update; the usual one-sided Count-Min error bound no longer applies without additional assumptions, and the deployment must declare which error guarantee (signed-additive vs. standard one-sided) it relies on through M and the release policy.

The transition circuit below certifies one committed epoch

$$B_t = \{(k_p, v_p)\}_{p=1}^{m}$$

against an old state commitment C_{old} and a new state commitment C_{new}. The circuit uses a database-style maintenance plan:

$$B \xrightarrow{\text{group by key}} G \xrightarrow{\text{compute counter ids}} U_{\text{ctr}} \xrightarrow{\text{group by counter}} A \xrightarrow{\text{sparse patch}} S_t.$$

The first grouping avoids repeated hash computation for duplicate keys. The second grouping is still required because different keys may collide in the same Count-Min counter.

Count-Min Update Circuit.
Public input. Epoch descriptor D_t, batch commitment C_{B_t}, old state commitment C_{old}, new state commitment C_{new}, and metadata M fixing d, w, the hash profile, counter representation, commitment scheme, and public capacities.
Private witness. Batch table B, key-sorted table $B^{\uparrow k}$, grouped-key table G, counter-update table U_{ctr}, counter-sorted table $U_{\text{ctr}}^{\uparrow}$, sparse patch table A, old/new opening tables O, N, and the commitment-update witness ρ_Δ (the committed blinding difference for hiding commitments; backend-specific update witness otherwise; zero in the non-hiding additive case) used by the Step 7 group relation.
Output. A proof that C_{new} commits to the result of applying the Count-Min batch update to C_{old} using the committed batch C_{B_t}.
Verification target. For every Count-Min counter (i, b),

$$S_t[i, b] = S_{t-1}[i, b] + \sum_{p:\, h_i(k_p) = b} v_p.$$

5.3.1 Update Circuit.

Step 1: Bind the private batch to the public commitment. The circuit starts with a private batch table

$$B[p] = (a_p, k_p, v_p),$$

where $a_p \in \{0, 1\}$ is an active flag and p is a local row index. The circuit verifies that the active rows of B open from the public batch commitment C_{B_t}, that those rows match the upload-log slice declared by the public epoch descriptor D_t (so the epoch's input is pinned to the same records committed earlier on ULog), and that inactive rows are canonical dummy rows. This prevents the prover from inventing, dropping, or modifying records while preserving the same public batch commitment.

Step 2: Bring equal keys together. The prover supplies a key-sorted table $B^{\uparrow k}$. The circuit checks

$$B^{\uparrow k} \equiv B$$

as multisets, and verifies that $B^{\uparrow k}$ is sorted by key with inactive rows placed last. After this step, all records with the same key appear in one contiguous segment.

This is a standard database operation: it is a verified ORDER BY key view of the committed batch, with a permutation check ensuring that no row has been inserted or lost.

Step 3: Aggregate values for each distinct key. The circuit scans $B^{\uparrow k}$ and aggregates each equal-key segment. The output is a grouped-key table

$$G[u] = (\alpha_u, k_u, V_u),$$

where $\alpha_u \in \{0, 1\}$ is an active flag and

$$V_u = \sum_{p:\, k_p = k_u} v_p.$$

Thus G contains one active row per distinct key in the epoch.

This preaggregation is sound because Count-Min updates are additive and commutative. For any key k,

$$\mathsf{Update}(k, v_1) \circ \mathsf{Update}(k, v_2) = \mathsf{Update}(k, v_1 + v_2).$$

Therefore, replacing many records with the same key by one grouped row with the summed value preserves the final sketch state. It also reduces the number of native hash computations from md to $U_{\max} d$, where $U_{\max}$ is the public padded capacity of G.

Step 4: Compute the counters updated by each distinct key. For each active grouped row $G[u] = (1, k_u, V_u)$, the circuit computes the d Count-Min bucket locations

$$b_{u,i} = h_i(k_u), \qquad i \in [d],$$

and range-checks

$$0 \le b_{u,i} < w.$$

The grouped key k_u contributes the same aggregate value V_u to one counter in each sketch row. Equivalently, the circuit forms a counter-update table

$$U_{\mathrm{ctr}}[u, i] = (a_{u,i}, k_u, i, b_{u,i}, V_u),$$

where $a_{u,i} = \alpha_u$. Inactive grouped-key rows produce canonical dummy counter-update rows.

If delete-aware updates are enabled, the grouped value is signed:

$$V_u = \sum_{p:\, k_p = k_u} \mathrm{op}_p \, v_p, \qquad \mathrm{op}_p \in \{+1, -1\}.$$

The rest of the circuit is unchanged; the counter range checks simply enforce the declared signed-counter representation.

Step 5: Bring updates to the same counter together. Different keys may hash to the same Count-Min counter. Therefore, the prover supplies a counter-sorted table $U^{\uparrow}_{\mathrm{ctr}}$, sorted by the counter identifier (i, b). The circuit verifies

$$U^{\uparrow}_{\mathrm{ctr}} \equiv U_{\mathrm{ctr}}$$

as multisets, including the active flag, key, row index, bucket, and value columns. It also verifies that $U^{\uparrow}_{\mathrm{ctr}}$ is lexicographically sorted by (i, b), with inactive rows placed last.

After this step, all contributions to the same sketch counter appear in one contiguous segment. The first grouping, in Step 3, removes repeated work from duplicate input keys. This second grouping handles hash collisions between distinct keys.

Step 6: Sum the contributions for each touched counter. The circuit performs segment aggregation over $U^{\uparrow}_{\mathrm{ctr}}$. For each contiguous segment with the same counter identifier (i, b), it computes

$$\Delta_{i,b} = \sum_{u:\, h_i(k_u) = b} V_u.$$

The output is a sparse patch table

$$A[z] = (\beta_z, i_z, b_z, \Delta_{i_z, b_z}),$$

where $\beta_z \in \{0, 1\}$ is an active flag. Each active row of A corresponds to one touched Count-Min counter, and $\Delta_{i,b}$ is the net amount by which the epoch changes that counter. The circuit additionally checks that the active rows of A are sorted by (i, b) and contain no duplicate counter identifiers (each touched counter appears at most once), and that inactive rows are canonical dummies; this normal form is what lets the commitment-update relation of Step 7 treat Δ as a well-defined sparse vector.

This is the only place where we need the term "delta": it denotes the net counter change after all duplicate-key aggregation and hash-collision aggregation have already been checked.

Step 7: Apply the sparse patch to the committed state. The prover supplies old and new opening tables O and N for the touched counters listed in A. For every active row $(i, b, \Delta_{i,b}) \in A$, the circuit checks

$$N[i, b] = O[i, b] + \Delta_{i,b}.$$

It also verifies that $O[i, b]$ opens correctly against C_{old}, that $N[i, b]$ opens correctly against C_{new}, and that all counters stay within the declared integer range. All signed counter additions are range-checked in the declared integer domain, and the field modulus of the backend is chosen large enough that valid integer updates — including the worst-case negative-going accumulation under signed Count-Min — do not wrap modulo $\mathbb{F}$, so the integer arithmetic the lemma certifies is faithful to the field arithmetic the circuit checks.

The opening checks above bind the touched cells correctly, but they do not by themselves prove that all *untouched* counters in S are unchanged — without an additional binding, a malicious prover could mutate untouched cells and still pass the touched-cell openings. The sparse-patch check must therefore be paired with a commitment-level update argument proving that C_{new} differs from C_{old} only on the coordinates listed in A. With a homomorphic (possibly hiding) vector commitment, this is the single group relation

$$C_{\mathsf{new}} = C_{\mathsf{old}} + \mathsf{Com}(\Delta;\, \rho_\Delta),$$

where Δ is the sparse vector represented by A and ρ_Δ is the committed blinding difference between the old and new state commitments (or, equivalently, the backend-specific update witness; under a non-hiding additive commitment ρ_Δ collapses to zero). The relation is mandatory, not optional, and it avoids opening or recomputing the entire sketch state. With a Merkle-style commitment the circuit must instead either certify unchangedness for every coordinate not in A or invoke a vector-commitment update proof with the equivalent guarantee.

Step 8: Check metadata compatibility and expose the history leaf. Finally, the circuit checks that the metadata M fixes the declared Count-Min parameters and circuit configuration: d, w, hash profile, counter representation, commitment scheme, and public capacities.

The resulting transition proof is then exposed as the leaf payload consumed by the recursive history-composition layer of §4.3. Thus, local transition soundness is handled by the Count-Min update circuit, while multi-epoch history soundness is handled by the root proof $R^{(t)}$.

Running example. Consider the epoch

$$(A, 100),\ (B, 50),\ (A, 200),\ (C, 75),\ (B, 30),\ (D, 40),$$

with parameters $d = 2$, $w = 4$ and the hash profile

key	h_0	h_1
A	0	3
B	1	0
C	2	2
D	0	2

Step 3 collapses duplicate keys into the grouped values $A \mapsto 300$, $B \mapsto 80$, $C \mapsto 75$, $D \mapsto 40$, saving one hash evaluation per row for A and B. Step 4 expands each grouped key into its $d = 2$ counter targets. The hash profile intentionally produces two distinct-key collisions: A and D both land in counter $(0, 0)$, while C and D both land in counter $(1, 2)$. Step 6 resolves these by additive sum, producing the sparse patch

$$\Delta_{0,0} = 340, \quad \Delta_{0,1} = 80, \quad \Delta_{0,2} = 75, \quad \Delta_{1,0} = 80, \quad \Delta_{1,2} = 115, \quad \Delta_{1,3} = 300,$$

so each collision is reconciled inside the sketch rather than overwriting the earlier contribution. Figure 5 traces the same epoch end-to-end through the eight-step circuit.

5.3.2 Circuit Checks. The eight-step Count-Min update relation decomposes into four checks. We state them at the level of relations rather than backend-specific gates; concrete arithmetizations can realize them using sorting, permutation, lookup, product-check, or AIR-style constraints.

Key-grouping check. The prover supplies a grouped-key table G. The circuit verifies that G is exactly the aggregation of the committed batch by key:

$$V_u = \sum_{p:\, k_p = k_u} v_p.$$

This check binds the grouped rows to C_{B_t} and prevents the prover from changing the multiplicity or total value of any key.

Counter-target check. For each grouped row (k_u, V_u) and each sketch row $i \in [d]$, the circuit verifies that the claimed counter identifier is

$$(i, b_{u,i}) = (i, h_i(k_u)), \qquad 0 \le b_{u,i} < w.$$

The resulting counter-update rows carry contribution V_u.

Counter-aggregation check. The prover supplies a sparse patch table A. The circuit verifies that each patch entry is the sum of all counter-update rows mapped to the same counter:

$$\Delta_{i,b} = \sum_{u:\, h_i(k_u) = b} V_u.$$

This check is necessary because distinct keys may collide in the same Count-Min counter.

Sparse-patch check. For each active row $(i, b, \Delta_{i,b}) \in A$, the circuit verifies that the old and new committed states differ by exactly this amount:

$$S_t[i, b] = S_{t-1}[i, b] + \Delta_{i,b}.$$

The check is performed for the touched counters listed in A and is paired with the mandatory commitment-level update argument (Step 7) that proves all untouched counters remain unchanged: under a homomorphic-additive vector commitment the single group relation $C_{\text{new}} = C_{\text{old}} + \mathsf{Com}(\Delta;\, \rho_\Delta)$ (with ρ_Δ the committed blinding difference for hiding commitments, collapsing to zero in the non-hiding case) binds the entire state.

Together with this commitment-level update argument, these four checks imply that for every counter (i, b),

$$S_t[i, b] = S_{t-1}[i, b] + \sum_{p:\, h_i(k_p) = b} v_p,$$

which is exactly the Count-Min batch update.

5.4 Greenwald–Khanna Circuit Design

We instantiate Greenwald–Khanna (GK) using the bounded-fusion variant adopted by S^3 [36]. This is not the original two-field GK tuple. Each active tuple has the form

$$(e, g_1, g_2, \Delta_1, \Delta_2, t),$$

where e is the representative value, g_1 and g_2 encode the rank mass immediately around e, Δ_1 and Δ_2 encode rank uncertainty from neighbouring ranges, and t is the insertion timestamp used to make fusions deterministic. The table is sorted by e, padded with dummy rows, and committed at a public capacity

$$s'(N) = \left\lceil 2s \ln\left(\frac{N}{s} + 2\right) + 2 \right\rceil,$$

where N is the cumulative number of inserted records. The parameter s controls the rank-error tolerance; the running capacity $s'(N)$ grows only mildly with N/s.

The transition circuit certifies the bounded-GK batch update used in the MPC construction: insert the new epoch values into the old sorted tuple table, initialize their rank-uncertainty fields from neighbouring old tuples, perform a fixed number of adjacent-tuple fusion rounds, compact the result, and recommit the bounded table. We use this as the declared GK update algorithm. We do not rely on GK being a commutative mergeable monoid over arbitrary summaries; for update proofs, it is enough that the declared batch-fusion procedure is deterministic and has a public capacity bound.

Greenwald–Khanna Update Circuit.
Public input. Epoch descriptor D_t, batch commitment C_{B_t}, old state commitment C_{old}, new state commitment C_{new}, old population N_{t-1}, new population $N_t = N_{t-1} + m$, and metadata M fixing s, the capacity function $s'(N)$, value representation, commitment scheme, timestamp convention, and public table capacities.
Private witness. Batch table B, old GK table S_{old}, sorted union table U, initialized union table $U^{(0)}$, fusion-round tables $U^{(1)}, \ldots, U^{(L)}$, compacted new table S_{new}, and commitment-opening witnesses.

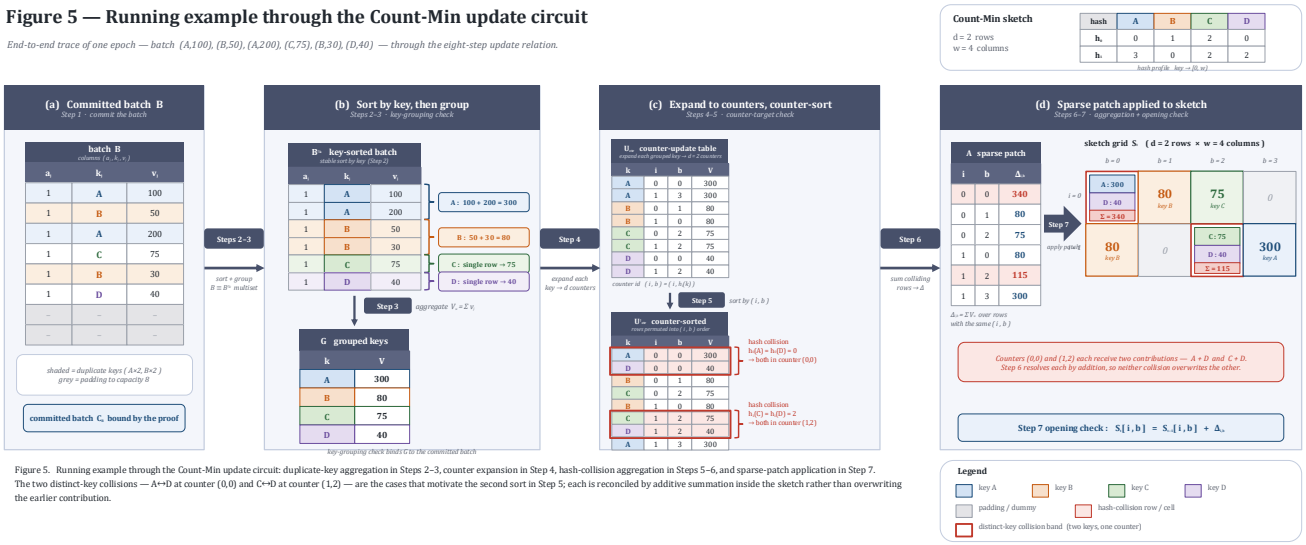


Figure 5. Running example through the Count-Min update circuit: duplicate-key aggregation in Steps 2–3, counter expansion in Step 4, hash-collision aggregation in Steps 5–6, and sparse-patch application in Step 7. The two distinct-key collisions — A↔D at counter (0,0) and C↔D at counter (1,2) — are the cases that motivate the second sort in Step 5; each is reconciled by additive summation inside the sketch rather than overwriting the earlier contribution.

Figure 5: Running example through the Count-Min update circuit.

Output. A proof that C_{new} commits to the result of applying the declared bounded-GK batch update to C_{old} using the committed batch C_{B_t}.
Verification target. The new committed state is the bounded-GK summary obtained by inserting all values in B_t into the old summary, initializing the new tuple fields according to the declared bounded-GK rule, applying the fixed fusion schedule, and compacting to public capacity $s'(N_t)$. The capacities $s'(N_{t-1}), s'(N_t)$ and the fusion-round count L are public deployment parameters fixed in M; the circuit only checks consistency with those public values rather than evaluating $s'(N) = \lceil 2s \ln(N/s + 2) + 2 \rceil$ in-circuit.
Postcondition. The compacted table S_{new} is sorted by (e, t), has total rank mass N_t, satisfies the declared bounded-GK tuple invariant, and is the deterministic result of applying the public fusion schedule of L parity-alternating adjacent rounds to the initialised union table $U^{(0)}$.

5.4.1 *Update Circuit.*

Step 1: Bind the old state and the private batch. The circuit starts with a private batch table

$$B[p] = (a_p, e_p),$$

where $a_p \in \{0, 1\}$ is an active flag, e_p is the uploaded value, and p is a local row index. It verifies that active rows of B open from the public batch commitment C_{B_t}, that those rows match the upload-log slice declared by the public epoch descriptor D_t, and that inactive rows are canonical dummies. The circuit also verifies that the old tuple table

$$S_{\text{old}}[z] = (a_z, e_z, g_{1,z}, g_{2,z}, \Delta_{1,z}, \Delta_{2,z}, t_z)$$

opens from C_{old}, is sorted by (e, t), and is padded to the public old capacity $s'(N_{t-1})$. These checks bind the transition to the committed old state and the committed epoch.

Step 2: Insert the epoch values into the old tuple order. Let $m_t = \ell_t - \ell_{t-1}$ be the public number of active uploaded values in this epoch (read off the descriptor D_t), and let $m_{\max}$ be the padded batch capacity, so the active flags of B satisfy $m_t = \sum_{p=1}^{m_{\max}} a_p$. Define the active ordinal of batch row p as $\text{ord}_p = \sum_{h \le p} a_h$, computed in-circuit from the active flags. The prover supplies a sorted union table U of capacity

$$M = s'(N_{t-1}) + m_{\max}.$$

The active rows of U consist of all active old tuples and one fresh tuple for each active batch value. A fresh tuple for $B[p] = (1, e_p)$ has timestamp $t_p = N_{t-1} + \text{ord}_p$ — the active ordinal, not the padded row index, so inactive padding rows do not introduce skipped or misleading timestamps — and placeholder rank fields:

$$(e_p, \bot, \bot, \bot, \bot, N_{t-1} + \text{ord}_p).$$

The circuit verifies two facts: first, U is a multiset union of the old tuples and the fresh batch tuples; second, U is sorted by the lexicographic key (e, t) — with the timestamp t acting as the strict deterministic tie-breaker whenever two tuples carry the same value e — and inactive rows are placed last. Because old tuples and fresh tuples carry disjoint timestamp ranges (old timestamps $< N_{t-1} + 1$, fresh timestamps $\in [N_{t-1} + 1, N_{t-1} + m_t]$ via the active ordinal), the (e, t) ordering is unique, so the sorted-union witness is canonical even when several batch values duplicate an existing e.

Step 3: Initialize the new tuple fields. For every fresh tuple, the bounded-GK rule sets

$$g_1 = 0, \qquad g_2 = 0.$$

The uncertainty fields are copied from the nearest old tuple on each side. Let $R^+(r)$ be the nearest active old tuple after row r in U, and let $R^-(r)$ be the nearest active old tuple before row r. For a fresh tuple at row r, the circuit checks

$$\Delta_1[r] = \begin{cases} R^+(r).g_1 + R^+(r).\Delta_1, & \text{if } R^+(r) \text{ exists,} \\ 0, & \text{otherwise,} \end{cases}$$

and

$$\Delta_2[r] = \begin{cases} R^-(r).g_2 + R^-(r).\Delta_2, & \text{if } R^-(r) \text{ exists,} \\ 0, & \text{otherwise.} \end{cases}$$

Old tuples keep their original fields. The initialized table is denoted $U^{(0)}$.

This step can be checked by prefix-copy and suffix-copy certificates: one pass propagates the nearest preceding old tuple information, and one pass propagates the nearest following old tuple information. The public shape of the check depends only on M, not on the private locations of the inserted values.

Step 4: Fix the fusion threshold and number of rounds. After the epoch is inserted, the new population is

$$N_t = N_{t-1} + m_t,$$

using the active count m_t of Step 2 rather than the padded batch capacity $m_{\max}$. The fusion threshold is

$$\theta_t = \left\lfloor \frac{N_t}{s} \right\rfloor.$$

The declared bounded-GK update is parameterised by a public fusion-round count L, fixed in M at deployment time. The proof certifies the deterministic result after exactly L public rounds applied to the initialised union table $U^{(0)}$; it does not by itself claim that all eligible fusions are exhausted. Our default is $L = \lceil \log M \rceil$ with $M = s'(N_{t-1}) + m_{\max}$, following the geometric-reduction argument of the S^3 bounded-fusion construction; a deployment that wants the stronger guarantee that all eligible fusions are exhausted must either choose L large enough for that policy or cite (or prove) the corresponding round bound. Treating L as a public parameter rather than a derived constant lets the deployment tune this trade-off without changing the circuit shape (which depends only on L and M, both fixed in M).

Step 5: Certify adjacent tuple fusions. The prover supplies tables

$$U^{(1)}, U^{(2)}, \ldots, U^{(L)}$$

representing the table after each fusion round. Each round applies a fixed adjacent-pair schedule, alternating parity so that every adjacent pair is checked across the two passes of a round.

For two adjacent active tuples

$$R_1 = (e_1, g_{1,1}, g_{2,1}, \Delta_{1,1}, \Delta_{2,1}, t_1)$$

and

$$R_2 = (e_2, g_{1,2}, g_{2,2}, \Delta_{1,2}, \Delta_{2,2}, t_2),$$

the circuit verifies exactly one of the following outcomes.

If $t_1 > t_2$, then the newer left tuple may be fused into the right tuple when

$$g_{1,1} + g_{2,1} + g_{1,2} + \Delta_{1,2} + 1 \le \theta_t.$$

In that case, R_1 becomes dummy and R_2 is updated by

$$g'_{1,2} = g_{1,1} + g_{2,1} + g_{1,2} + 1,$$

while the representative value, timestamp, and uncertainty fields of R_2 remain those of R_2.

Symmetrically, if $t_2 > t_1$, then the newer right tuple may be fused into the left tuple when

$$g_{1,2} + g_{2,2} + g_{2,1} + \Delta_{2,1} + 1 \le \theta_t.$$

In that case, R_2 becomes dummy and R_1 is updated by

$$g'_{2,1} = g_{1,2} + g_{2,2} + g_{2,1} + 1.$$

If the relevant inequality fails, or if one of the two rows is inactive, both rows are copied forward unchanged. These checks certify the same local FuseTuple rule as the declared bounded-GK update.

Step 6: Compact and recommit the bounded state. After the L fusion rounds, the prover supplies a compacted table S_{new}. The circuit verifies that S_{new} is the deterministic order-preserving compaction of the active rows of $U^{(L)}$, padded to the public new capacity s'_t, where s'_t is the deployment parameter corresponding to $s'(N_t)$ and is fixed in M. The deployment chooses s'_t according to the closed form

$$s'(N_t) = \left\lceil 2s \ln\left(\frac{N_t}{s} + 2\right) + 2 \right\rceil,$$

but the circuit treats s'_t as public metadata rather than evaluating the logarithm in-circuit. The circuit then verifies that S_{new} opens to the published commitment C_{new}. This is a whole-state replacement check rather than a sparse additive patch: GK is an ordered summary whose correctness is global, not a fixed array of independent counters.

Step 7: Check metadata compatibility and expose the history leaf. Finally, the circuit checks that the metadata M fixes the declared GK variant, including s, the capacity function $s'(N)$, tuple layout, timestamp convention, fusion threshold, fusion schedule, commitment scheme, and public table capacities. The resulting transition proof is exposed as the leaf payload consumed by the recursive history-composition layer of §4.3.

Running example. Take the old committed state S_{old} with four active tuples $(e, g_1, g_2, \Delta_1, \Delta_2, t)$,

$$(3, 1, 1, 0, 1, 5),\ (7, 2, 3, 1, 0, 10),\ (12, 1, 1, 2, 3, 15),\ (18, 1, 0, 0, 2, 2),$$

the epoch $B_t = (1, 5, 15, 6)$ with timestamps $17, 18, 19, 20$, $N_{t-1} = 16$, and toy fusion threshold $\theta_t = \lfloor N_t/s \rfloor = 2$. (The capacity $s'(N_t)$ padding is suppressed for readability.)

Step 2 inserts the four fresh tuples into the sorted union, producing the value order $1, 3, 5, 6, 7, 12, 15, 18$ with $\perp$ placeholders on each fresh row. Step 3 fills those placeholders by copying from the nearest old neighbour on each side: for instance the fresh row $e = 5$ sits between old tuples 3 and 7, so

$$\Delta_1(5) = g_1(7) + \Delta_1(7) = 2 + 1 = 3, \quad \Delta_2(5) = g_2(3) + \Delta_2(3) = 1 + 1 = 2,$$

and the fresh row $e = 6$ inherits the same neighbours.

Step 5 then applies the parity-alternating fusion schedule. Two fresh–old pairs qualify under $\theta_t = 2$: for $(1, 3)$, the newer-left rule gives $g_{1,1} + g_{2,1} + g_{1,2} + \Delta_{1,2} + 1 = 0 + 0 + 1 + 0 + 1 = 2 \le \theta_t$, so $e = 1$ fuses right into $e = 3$ with the update $g_1(3) \leftarrow g_{1,1} + g_{2,1} + g_{1,2} + 1 = 2$; the pair $(15, 18)$ is symmetric and yields $g_1(18) \leftarrow 2$. The middle fresh–fresh pair $(5, 6)$ instead fails the newer-right inequality $g_{1,2} + g_{2,2} + g_{2,1} + \Delta_{2,1} + 1 = 0 + 0 + 0 + 2 + 1 = 3 > \theta_t$ and both rows are copied forward unchanged. This is the pedagogically important case: the threshold inequality, not the prover, decides what fuses. Step 6 stably compacts the two dummy rows, giving a six-tuple S_{new} that is then padded and recommitted. Figure 6 traces the five intermediate tables side-by-side.

Figure placeholder. End-to-end trace of the running example through the bounded-GK update circuit, drawn as five horizontally arranged tuple tables connected by labelled arrows. Every panel uses the same column layout $(e, g_1, g_2, \Delta_1, \Delta_2, t)$ so the reader can scan one row across the pipeline. **(a) Old state** S_{old}: four active rows $e \in \{3, 7, 12, 18\}$ with their committed rank fields; a few greyed dummy rows below indicate padding to $s'(N_{t-1})$. **(b) Sorted union** U: the four fresh rows $e \in \{1, 5, 6, 15\}$ interleaved among the old rows in order 1, 3, 5, 6, 7, 12, 15, 18, with the $\perp$ cells drawn in italic grey and the timestamp column highlighted to make the (e, t) sort visible. A small $\equiv$ badge on the arrow from (a) to (b) marks the multiset check of Step 2. **(c) Initialized union** $U^{(0)}$: same row layout as (b) with the $\perp$ cells replaced. Two dashed back-arrows on the fresh row $e = 5$ point at old rows 3 (annotated "$\Delta_2 = g_2(3)+\Delta_2(3) = 2$") and 7 (annotated "$\Delta_1 = g_1(7)+\Delta_1(7) = 3$"); a second pair of dashed arrows annotates the fresh row $e = 15$. This visualises the prefix/suffix-copy initialization of Step 3. **(d) Post-fusion table** $U^{(L)}$: rows $e = 1$ and $e = 15$ are crossed out (now dummy) and the cells $g_1(3)$, $g_1(18)$ are recoloured to show their new value 2. Two green check marks sit on the successful pairs $(1, 3)$ and $(15, 18)$ annotated with "$0+0+1+0+1 = 2 \le \theta_t$"; a red "no-fuse" badge on the middle pair $(5, 6)$ carries the failing inequality "$0+0+0+2+1 = 3 > \theta_t$" to emphasise that the threshold check gates each pair locally. **(e) Compacted new state** S_{new}: six active rows in original relative order, padded to $s'(N_t)$, with an outgoing arrow to the commitment C_{new} on the right.

Figure 6: Running example through the bounded-GK update circuit: sorted insertion of the epoch (Step 2), prefix/suffix-copy initialization of fresh tuples (Step 3), parity-alternating fusion under threshold $\theta_t = 2$ (Step 5), and deterministic order-preserving compaction with recommit (Step 6). The unfused pair $(5, 6)$ is drawn explicitly so that the reader sees the threshold inequality, rather than the prover, deciding which adjacent pairs fuse.

Summary. The GK transition circuit proves five logical claims. First, the old state and epoch rows are bound to their public commitments. Second, the union table is the sorted insertion of the epoch values into the old tuple table. Third, the rank-uncertainty fields of fresh tuples are initialized from neighbouring old tuples according to the bounded-GK rule. Fourth, the fixed fusion schedule correctly applies the local FuseTuple rule under threshold $\theta_t = \lfloor N_t/s \rfloor$. Fifth, the compacted bounded table is exactly the new committed state. Together these claims certify that C_{new} is the result of the declared bounded-GK batch update.

5.4.2 Circuit Checks. The update relation above can be summarized by four checks. We state them at the relation level, independent of any particular proof backend.

Insertion-order check. The circuit verifies that the sorted union table contains exactly the old tuples and the new batch tuples, and that it is ordered by (e, t). This replaces the data-dependent insertion positions of the plaintext algorithm with a single ordered witness table.

Initialization check. The circuit verifies that each new tuple has $g_1 = g_2 = 0$, and that Δ_1, Δ_2 are copied from the nearest old tuple on the right and left, respectively. This is the certificate analogue of the suffix-copy and prefix-copy initialization used in the bounded-GK streaming protocol.

Fusion check. The circuit verifies a public number of adjacent fusion rounds. Each pair either remains unchanged or is fused according to the timestamp order and the threshold inequalities above. Since the number of rounds is public, the circuit shape does not reveal how many fusions actually occurred.

Replacement check. The circuit verifies that the new committed state is the deterministic order-preserving compaction of the active rows of the final fusion table $U^{(L)}$, padded to the public capacity s'_t. Unlike Count-Min, there is no per-cell delta vector; the certificate proves a valid replacement of the whole bounded summary.

5.4.3 Rank Query Circuit. A GK rank query asks for the approximate rank of a query value q. In zkSketch, queries are answered against a closed sketch version: if an epoch is open when the query arrives, it is first closed and proved. Therefore, the query circuit reads only the committed GK state. If a deployment keeps an explicit unmerged buffer, the buffer contribution can be added by the same comparison-and-sum check used in S^3.

The committed GK table is extended with two sentinel tuples $e_0 = -\infty$ and $e_{|S_t|+1} = +\infty$ (both with zero rank mass and dummy timestamps), so the bracketing pair is always well-defined for any q including boundary cases $q < e_1$ and $q \ge e_{\max}$. The prover opens the committed table and supplies selector bits identifying the adjacent tuple pair that brackets q. Let i be the selected index such that

$$e_i \le q < e_{i+1},$$

with the convention $e_0 \le q$ always and $q < e_{|S_t|+1}$ always. The circuit verifies the one-hot selector and the two comparisons above. It also verifies the prefix sums

$$P_i = \sum_{z \le i} \big(g_{1,z} + g_{2,z} + 1\big).$$

Then it computes

$$r_{\min} = P_i - \Delta_{2,i} - g_{2,i},$$

and

$$r_{\max} = P_i + \Delta_{1,i+1} + g_{1,i+1}.$$

The raw decoded rank estimate is the rational pair

$$dec = (r_{\min} + r_{\max},\ 2),$$

representing the midpoint $(r_{\min} + r_{\max})/2$; the circuit keeps the numerator and denominator separately to avoid forcing field division, and the integer-rank encoding (floor, ceiling, or banker's rounding) is delegated to the release policy. When the release policy of §3.3 is the identity, $ans = dec$ is published as the rational pair and the proof certifies it directly. When *policy* is non-trivial (deterministic rounding, suppression below a threshold, or noise from a committed seed ρ), dec remains a private witness and the proof certifies

$ans = policy(dec; \rho)$ against the committed state version, in line with the dec/ans split of Property 3 of §3.5.

Greenwald–Khanna Rank Query Circuit.
Public input. State commitment C_{state}, query value q, metadata M (which fixes the release policy), and published answer *ans*.
Private witness. Opened GK table S_t, opening witnesses, prefix-sum witness P, selector bits for the bracketing pair, intermediate values $r_{\min}, r_{\max}$, the decoded estimate *dec*, and (when the release policy is non-trivial) the opening of the committed release seed ρ.
Verification target. The opened table is the committed GK state, the selected adjacent tuple pair brackets q, and the circuit computes

$$n = (P_i - \Delta_{2,i} - g_{2,i}) + (P_i + \Delta_{1,i+1} + g_{1,i+1}),$$

proving

$$dec = (n,\ 2)$$

as the rational pair encoding the midpoint $n/2$. For identity release policy the circuit additionally proves $ans = dec$; for non-trivial *policy* it proves $ans = policy(dec; \rho)$ against the committed seed.
Checks. The circuit checks the state opening, tuple ordering, one-hot bracket selection, prefix-sum recurrence, the formulas for $r_{\min}$ and $r_{\max}$, the decoded estimate *dec*, and the identity or release-policy binding from *dec* to *ans*.

5.4.4 Remark on Mergeability. The bounded-GK variant above is suitable for our update circuit because it gives a deterministic, capacity-bounded batch-fusion procedure. In this subsection, "merge" means merging a batch of new values into one existing GK summary, as in the S^3 protocol. We do not require the stronger property that two arbitrary independently maintained GK summaries form a commutative mergeable monoid. If a deployment needs cross-shard GK merging, it should either use a separately declared GK summary-merge algorithm and prove that relation, or close each shard to a committed epoch sequence and combine histories at the framework layer. The transition circuit here only assumes the bounded batch-update procedure specified above.

5.5 Correctness and Cost Summary

Both representative circuits avoid replaying the per-record sketch update inside the circuit and instead certify the batch-effect relation directly. We state the resulting correctness guarantees as two lemmas and summarise the per-family costs in Table 4. The circuits are *optimised relative to the strawman* (no per-record state materialisation, no full-state recommit per record, key preaggregation before hashing for Pattern A linear) and *backend-pluggable* (the sort/segment gates may be specialised with lookup-argument routing, wide-output hashing, or lookup-backed sortedness without changing the verification target); they are not claimed to be globally optimal.

Lemma 1 (Count-Min transition correctness). *If the Count-Min transition circuit of §5.3 accepts on public input* $(D_t, \mathsf{Com}(S_{t-1}), C_{B_t}, \mathsf{Com}(S_t), M)$ *under Assumption 1 of §3.4, then for every counter* (i, b),

$$S_t[i,b] \;=\; S_{t-1}[i,b] \;+\; \sum_{p:\, h_i(k_p)=b} v_p,$$

Table 4: Per-family circuit correctness invariants and dominant cost lines. Costs are stated in the certificate's witness/check count; constant-factor backend specialisations (lookup-argument routing, wide-output hashing, lookup-backed sortedness) are deferred to the per-batch empirical evaluation of §6.

Sketch	Correctness invariant (Lemma)	Dominant cost
Count-Min	exact signed linear counter update (Lemma 1)	sort B (m), hash
Greenwald–Khanna	declared L-round bounded fusion (Lemma 2)	sort union ($M =$
HyperLogLog (App.)	max-rebind of touched registers	hash m, sort U,
SpaceSaving (App.)	top-s retention via threshold-residual	merge $\lvert S\rvert + m$, t

where the sum is over the active rows of B_t that open from C_{B_t} and match the upload-log slice declared by D_t.

Proof sketch. By Steps 1–3 of §5.3, the grouped-key table G is the exact aggregation by key of the rows of B opened from C_{B_t} (input binding via $\mathsf{SliceOK}(D_t, B_t)$ plus permutation/sortedness/segment-sum checks). By Step 4, the counter-update table U_{ctr} contains, for every active grouped key k_u and every $i \in [d]$, the contribution $(k_u, i, h_i(k_u), V_u)$. By Steps 5–6, the sparse patch table A contains, for every touched counter (i, b), the sum $\Delta_{i,b} = \sum_{u:\, h_i(k_u)=b} V_u$. Combining with $V_u = \sum_{p:\, k_p=k_u} v_p$ gives the claimed sum. By Step 7, the touched cells $N[i,b] = O[i,b] + \Delta_{i,b}$ are bound to $\mathsf{Com}(S_t)$ via opening, and the mandatory commitment-update relation $C_{\text{new}} = C_{\text{old}} + \mathsf{Com}(\Delta; \rho_\Delta)$ binds every untouched cell to be unchanged. Together these force $S_t[i,b]$ to equal the stated expression for every (i,b). □

Lemma 2 (Greenwald–Khanna transition correctness). *If the GK transition circuit of §5.4 accepts on public input* $(D_t, \mathsf{Com}(S_{t-1}), C_{B_t}, \mathsf{Com}(S_t)$ *under Assumption 1, then* $\mathsf{Com}(S_t)$ *opens to the deterministic order-preserving compaction of the active rows of $U^{(L)}$, padded to the public capacity s'_t, where $U^{(L)}$ is the result of applying the public L-round parity-alternating adjacent fusion schedule with threshold $\theta_t = \lfloor N_t/s \rfloor$ to the initialised union table $U^{(0)}$ built from the old summary and the m_t active batch values of B_t. The lemma certifies the declared bounded-fusion GK update; it does not by itself claim that all eligible fusions are exhausted unless the deployment chooses L large enough to guarantee exhaustion or cites a corresponding round bound.*

Proof sketch. By Step 1, S_{old} opens from $\mathsf{Com}(S_{t-1})$ and B opens from C_{B_t} matching D_t. By Steps 2–3, the initialised union $U^{(0)}$ is the deterministic insertion of the m_t active batch values into S_{old} with the declared placeholder-initialisation rule (prefix-/suffix-copy from nearest active old neighbour, active-ordinal timestamps). By Step 5, each $U^{(j+1)}$ is the deterministic application of one parity-alternating fusion round to $U^{(j)}$ under threshold θ_t; by induction over $j = 0, \ldots, L-1$, $U^{(L)}$ is the deterministic result of the declared L-round schedule. By Step 6, S_t is the deterministic order-preserving compaction of the active rows of $U^{(L)}$ padded to s'_t. By the whole-state opening of $\mathsf{Com}(S_t)$ in Step 6 and commitment binding, $\mathsf{Com}(S_t)$ opens to exactly this state. □

The lemmas combine with Property 1 of §3.5 to discharge update soundness for the four families under their declared algorithms;

query-prefix soundness (Property 3) additionally invokes the decode circuits of §5.3 and §5.4.

5.6 Coverage and Other Sketches

The remaining two families fill the same seven slots. HyperLogLog (Pattern A max with a leading-run hash) is in Appendix C; SpaceSaving (Pattern B with top-s retention via the threshold-residual certificate, in place of GK's rank-interval fusion) is in Appendix D. The framework naturally supports sketches whose update is expressible as either a sparse per-target operator patch (Pattern A) or a public-capacity-bounded deterministic replacement schedule (Pattern B). Several additional summaries are plausible extensions when their compaction, centroid, or merge schedule is made deterministic and public-capacity bounded — counting Bloom variants, MinHash, KLL [17], t-digest, and the broader mergeable-summaries class [1] are candidates in this sense. KLL's randomised compaction in particular requires committing the compaction randomness in advance as part of the public hash profile, and t-digest needs its centroid invariants made circuit-checkable. Out of scope: sketches whose merge invariant is non-constant-depth, online learned summaries with adaptive structure, and sketches whose accuracy guarantee relies on randomness that cannot be committed in advance under a public hash profile.

6 Evaluation (in progress)

This section describes the evaluation plan. Empirical results are not included in this draft; they will appear in the camera-ready version. The analytical evaluation in Appendix B grounds the design choices in constraint-count derivations validated against published constraint-count microbenchmarks from prior ZK-database systems. A convincing SIGMOD/VLDB evaluation should validate the comparative analysis empirically and quantify each design choice's contribution; we organise the planned experiments around four research questions that mirror the contribution structure of §1.

6.1 Research Questions

- **RQ1 (Capability).** Does delta-certified maintenance preserve sketch accuracy and produce checkable proofs across all four families (CM, HLL, SS, GK) under representative workloads?
- **RQ2 (Scalability).** How do prover time, verifier time, proof size, and memory scale along each cost dimension — state size s, depth d, batch size m, batch rate b, audit-window length E — compared to optimised exact-ZK-index and zkVM baselines?
- **RQ3 (Pattern-A and Pattern-B optimisations).** How much does each circuit-level optimisation deliver on its target workload? Specifically: the hash-once-per-distinct-key and wide-output two-field split optimisations for HLL (Appendix C.2); the single-round retention and one-pass threshold scan for SpaceSaving (Appendix D.2); and the sparse-rebind commitment for the Pattern A array sketches CM and HLL.
- **RQ4 (Composition topology trade-off).** Where do the three composition topologies (linear chain, balanced binary tree, streaming forest) and the four maintenance policies (lazy, eager, smoothed, query-burst) of §4.3 sit on the velocity-vs-latency plane?

6.2 Experimental Setup

Implementation. The prototype targets a standard arithmetic-circuit backend in Rust, reusing standard gates for range checks, sorting, lookups, equality flags, and arithmetic aggregation, following the methodology of PoneglyphDB and VPJoin [2, 16]. We do not commit to a specific backend instantiation: the per-circuit gates of §5 are expressed in terms of standard ZK primitives (permutation arguments, lookup arguments, vector-commitment openings) that any modern arithmetic-circuit backend supports.

Implementation roadmap. The implementation is staged in two increments. *Stage 1 (per-family update circuits):* implement CM, HLL, SS, GK update circuits against a single-sketch baseline; validate against the comparative analysis (RQ1 and RQ2). *Stage 2 (per-family optimisations and composition topologies):* add the HLL and SpaceSaving optimisations of Appendices C.2 and D.2, the sparse-rebind commitment for Pattern A array sketches, and the three composition topologies with the four maintenance policies of §4.3; run RQ3 and RQ4.

Reproducibility plan. The implementation, benchmark scripts, datasets (synthetic generators + pointers to public real datasets), and per-experiment configuration files will be released under an open-source license at the time of submission. Evaluation tables will report median and 95%-quantile of five trials; we will include a per-experiment cost-model prediction column to validate the analytical model of Appendix B against measured numbers.

Workloads. (i) Frequency / range-frequency over filings/transactions (CM); (ii) distinct counts over high-cardinality keys (HLL); (iii) top-k heavy hitters (SS); (iv) quantile queries on numerical fields (GK); (v) release policies (thresholding, rounding, optional noisy release).

Datasets. Synthetic uniform, Zipfian, and Gaussian record streams to control skew and collision behaviour; real datasets matching the deployments (SEC filings or simulated regulatory data; on-chain transactions; network flow traces). Dataset choice exercises both high-cardinality and skewed regimes.

Baselines. (i) Naive exact ZK index (Merkle leaves); (ii) optimised exact ZK index (sorted-index commitment with lookups, in the style of PoneglyphDB and ZKSQL); (iii) optimised sketch (this paper); (iv) zkVM baselines (RISC0 and SP1) [29, 32]; (v) plaintext sketch engine (Apache DataSketches [3]) as a lower-bound reference. The `naive-zkSketch` strawman of §5 appears as a rhetorical reference point in the RQ3 discussion; its asymptotic cost is derived analytically in Appendix B rather than measured, because its $\Theta(mdH \log w)$ per-batch cost would dominate any practical sweep before reaching the regime where the optimised sketch operates. Reporting baselines that run out of memory or time as `OOM`/`TLE` rather than silently skipping them, following the convention of ZK-Value [34], makes the comparison's failure modes explicit.

Metrics. Circuit rows, custom-gate counts, lookup counts, prover time, witness-generation time, verification time, proof size, memory, state size, commitment-opening cost, query latency, batch update throughput, touched-cell count, approximation error. Per

optimisation (RQ3): HLL hash-invocation count with and without hash-once-per-distinct-key under workload skew; HLL per-key hash invocations with and without the wide-output two-field split; SpaceSaving constraint count under single-round retention; commitment-opening cost under sparse-rebind versus whole-state recommit on Pattern A array sketches.

6.3 RQ1 — Capability

RQ1 is answered by running each of the four families against workload classes (i)–(iv). For each family the experiment certifies four things: (a) the update circuit accepts on honest witnesses across the full sweep of s, d; (b) the decode circuit reproduces the plaintext approximate answer to within the family's stated error bound; (c) the recursive history composition of §4.3 commits an audit window of E batches with verifier-side cost matching the predicted $O(1)$ (root proof) or $O(\log E)$ (per-batch maintenance); (d) a tampered batch — 5% row substitution — is rejected by the verifier with overwhelming probability.

Takeaway for RQ1. Expected: zkSketch produces checkable proofs for every workload across the four families, and tampering with 5% of rows in any batch is detected via the commitment-binding chain. The approximation error matches the family's analytical bound (a sketch property, not a ZK one).

6.4 RQ2 — Scalability

RQ2 is answered by sweeping each cost dimension while holding the others fixed at the running-example default, mirroring the dimension-by-dimension panels used by zk-Value (Figure 3 of [34]). Specifically: $s \in \{2^{10}, \ldots, 2^{18}\}$ for state size; $d \in \{1, \ldots, 16\}$ for depth; $m \in \{2^6, \ldots, 2^{14}\}$ for batch size; $b \in \{1, \ldots, 10^3\}$ batches per hour for batch rate; $E \in \{1, \ldots, 10^4\}$ for audit-window length. For each sweep we report prover time, verifier time, proof size, and peak memory; the optimised exact-ZK-index and zkVM baselines provide the comparison.

Takeaway for RQ2. Expected: prover time scales linearly in m and T and sub-linearly in s (touched-cell-only commitment work); verifier time is $O(1)$ in E via the recursive history root; proof size is constant per batch (single succinct proof). The per-record update cost ratio against the optimised exact index grows with n in the optimised sketch's favour.

6.5 RQ3 — Pattern-A and Pattern-B optimisations

RQ3 is answered by ablating each per-family optimisation on the workload it targets, with the canonical per-family update circuit (Appendix C, Appendix D, §5.3, §5.4) as the reference and the `naive-zkSketch` strawman of §5 as the rhetorical upper bound.

HLL: hash-once-per-distinct-key (Appendix C.2): sweep Zipf parameter $\alpha \in \{0, 0.5, 1, 1.5\}$ on a batch of $m = 16{,}384$ keys; measure realised distinct-key count u and the per-batch hash-invocation count with and without key-pre-grouping. Predicted reduction from $|B_t|$ to $u_{\max}$ permutations per batch, with the largest absolute saving at high duplication ($\alpha \geq 1$).

HLL: wide-output two-field split (Appendix C.2): on the same workload, compare the default two-hash design against the single wide-output permutation with two-limb split; expected 2× reduction in per-(distinct-)key hash invocations.

SpaceSaving: single-round retention (Appendix D.2): contrast the canonical single-round top-s retention against a (hypothetical) padded multi-round eviction shape that would have been used for uniformity with Pattern B fusion; report the avoided $\Theta(\log(|G|))$ round factor in constraint count.

Sparse-rebind commitment (HLL Optimisation C of Appendix C.2, and the same idea applied to Count-Min): on the Pattern A array sketches, contrast the per-batch commitment-rebind cost under (i) whole-state recommit and (ii) sparse rebind with explicit equality-outside-the-patch, as the touched-register count varies from $\Theta(1)$ to $\Theta(s)$. Predicted crossover where sparse dominates whenever the touched count is asymptotically below the state size.

Takeaway for RQ3. Expected: each optimisation delivers its predicted hash-count or commitment-cost reduction at its activation conditions; the gap between the optimised sketch and `naive-zkSketch` approaches the analytical ∼ 80× derived in §5. Together these confirm that each per-family optimisation pays for itself in the regime where it activates.

6.6 RQ4 — Composition topology trade-off

RQ4 validates the design-space summary of §4.3 by measuring per-update cost and query latency under the three composition topologies (linear chain, balanced binary tree, streaming forest) and the four maintenance policies (lazy, eager, smoothed, query-burst). The sweeps fix CM as the per-batch update circuit — the topology choice is sketch-agnostic — and vary the topology and policy as orthogonal axes.

4-a. Insert-rate scaling on a single-core prover. Sweep batch rate $b \in \{1, 10, 10^2, 10^3\}$ batches/sec on one core; report per-update online cost for the chain (one merge), the balanced binary tree (the $\Theta(\log E)$ spine refresh at $E = 1024$), and the streaming forest (cascade amortised and worst-case). Expected: the chain wins at low rate, the streaming forest's amortised $O(1)$ wins at high rate; the binary tree sits in between under its known-E caveat.

4-b. Insert-rate scaling on a multi-core prover. Repeat 4-a on a 16-core machine. Expected: the chain stays single-threaded and saturates at the same rate as 4-a; the balanced binary tree and the streaming forest scale to parallelism, validating the parallelisation motivation of §4.3.

4-c. Query latency under lazy vs. eager peak chain. Sweep query rate q from 1/min to 1/sec while holding the insert rate at 10^2 batches/sec; measure query latency (from query arrival to verifier-checked root proof) and worst-case insert latency (cascade-aligned bursts). Expected: under lazy, query latency follows $O(\log t)$ and worst-case insert latency follows the cascade depth; under eager, query latency is $O(1)$ and worst-case insert latency picks up the cascade-rebuild surcharge. The crossover at which eager dominates matches the predicted query-to-insert ratio threshold of §4.3.

4-d. Smoothed and query-burst policies. Vary the smoothing rate $c \in \{1, 2, 4\}$ on the streaming forest under the smoothed policy of §4.4; measure insert tail latency and the query-mid-backlog drain

cost. Compare against the query-burst policy under the same insert rate, varying the available core count k. Expected: smoothing flattens insert tail latency at the cost of mid-cascade query drain; query-burst converts that drain from $\Theta(|Q_t|)$ sequential into $\Theta(\lceil \log_2 |Q_t| \rceil)$ on k cores, validating the policy choice of Example 4.1.

Takeaway for RQ4. Expected: the topology and maintenance policy each have a measurable crossover point on the batch-rate / query-rate plane, and the four-policy design of §4.4 covers the Pareto frontier for streaming workloads with unpredictable query arrival.

7 Related Work

MPC sketches and streaming ZK. S^3 [36] designs efficient sketch protocols in outsourced MPC and is the closest algorithmic precedent: its batch-update insight (sort, segmented prefix-sum/max, compact, top-s retain, GK fusion rounds) reappears throughout our certificate-checking pipeline. zkSketch differs on four dimensions: *trust model* (S^3 is multi-server semi-honest MPC; zkSketch is single-prover non-interactive ZK against malicious provers), *output shape* (secret shares vs. transferable proofs), *state* (fresh per-batch vs. committed and tree-bound), and *cost model* (MPC rounds and bandwidth vs. arithmetic-circuit constraint count and proof size). The algorithmic primitives transfer; the cryptographic regime is fundamentally different. Whisper [28] combines streaming, sketching, and silently verifiable proofs with multiple non-colluding servers; streaming ZK proofs provide foundations for low-space verifiers [7]; both target high-throughput streaming with multi-party trust and are not direct comparators for the dynamic single-prover regime.

Verification-over-computation as a technique. Mystique [35], zkCNN [22], and vCNN [20] apply VoC pointwise (Freivalds' algorithm [11]) to one or two operators each. zkSketch elevates the technique to a framework discipline organised by certificate shape, with the seven-slot interface and Pattern A/B dichotomy of §5 as the realisation.

Sketches and AQP. Count-Min, HyperLogLog, SpaceSaving, and GK are classic compact summaries [8, 10, 14, 24]; KLL [17] and the broader mergeable-summaries class [1] are natural extensions; Apache DataSketches packages many in production form [3]. zkSketch differs by adding committed sketch state, a maintained sketch-merge segment tree for verifiable history binding, and verifiable updates over a single untrusted prover.

Recursion, folding, and proof pipelines. Recursive proofs, folding, IVC, and accumulation [6, 18, 19, 30] carry validity across long computations. zkSketch treats them as first-class composition primitives: the three composition topologies of §4.3 place the linear chain, balanced binary tree, and streaming forest (with lazy / eager / smoothed peak-chain policies) in one design space and characterise the trade-offs against batch size, parallelism, and core count. Reckle Trees [26] apply a single Merkle-tree topology to a static rollup; modern zkVMs and proof-pipeline aggregators [25, 27, 29, 31, 32] adopt k-ary recursion for general-purpose computation. zkSketch is, to our knowledge, the first work to characterise these topologies for the specific workload of streaming sketch maintenance under non-interactive ZK.

References

[1] Pankaj K. Agarwal, Graham Cormode, Zengfeng Huang, Jeff M. Phillips, Zhewei Wei, and Ke Yi. 2012. Mergeable Summaries. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*. 23–34. https://doi.org/10.1145/2213556.2213562

[2] Anonymous Authors. 2026. VPJoin: Verifiable and Privacy-Preserving Multi-Way Join via Zero-Knowledge Proofs. Manuscript supplied by the authors.

[3] Apache Software Foundation. 2026. Apache DataSketches. https://datasketches.apache.org/ Accessed 2026-05-03.

[4] Jon Louis Bentley and James B. Saxe. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *Journal of Algorithms* 1, 4 (1980), 301–358.

[5] Nir Bitansky, Ran Canetti, Alessandro Chiesa, Shafi Goldwasser, Huijia Lin, Aviad Rubinstein, and Eran Tromer. 2017. The hunting of the SNARK. *Journal of Cryptology* 30, 4 (2017), 989–1066.

[6] Benedikt Bünz and Binyi Chen. 2023. ProtoStar: Generic Efficient Accumulation/Folding for Special-Sound Protocols. In *Advances in Cryptology – ASIACRYPT 2023*. 77–110. https://doi.org/10.1007/978-981-99-8724-5_3

[7] Graham Cormode, Marcel Dall'Agnol, Tom Gur, and Chris Hickey. 2024. Streaming Zero-Knowledge Proofs. In *39th Computational Complexity Conference (LIPIcs)*, Vol. 300. 2:1–2:66. https://doi.org/10.4230/LIPIcs.CCC.2024.2

[8] Graham Cormode and S. Muthukrishnan. 2005. An Improved Data Stream Summary: The Count-Min Sketch and Its Applications. *Journal of Algorithms* 55, 1 (2005), 58–75. https://doi.org/10.1016/j.jalgor.2003.12.001

[9] Amos Fiat and Adi Shamir. 1986. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*. Springer, 186–194.

[10] Philippe Flajolet, Eric Fusy, Olivier Gandouet, and Frederic Meunier. 2007. HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm. In *Analysis of Algorithms 2007*. 127–146. https://algo.inria.fr/flajolet/Publications/FlFuGaMe07.pdf

[11] Rūsiņš Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. *Information Processing 77, Proceedings of IFIP Congress 77* (1977), 839–842.

[12] S. Goldwasser, S. Micali, and C. Rackoff. 1985. The Knowledge Complexity of Interactive Proof-Systems. In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing, May 6-8, 1985, Providence, Rhode Island, USA*.

[13] Shafi Goldwasser, Silvio Micali, and Chales Rackoff. 2019. The knowledge complexity of interactive proof-systems. In *Providing sound foundations for cryptography: On the work of shafi goldwasser and silvio micali*. 203–225.

[14] Michael Greenwald and Sanjeev Khanna. 2001. Space-Efficient Online Computation of Quantile Summaries. In *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*. 58–66. https://doi.org/10.1145/375663.375670

[15] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 305–326.

[16] Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-Interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. *Proceedings of the ACM on Management of Data* 3, 1, Article 63 (2025). https://doi.org/10.1145/3709713

[17] Zohar Karnin, Kevin Lang, and Edo Liberty. 2016. Optimal Quantile Approximation in Streams. In *Proceedings of the 57th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. 71–78. https://doi.org/10.1109/FOCS.2016.17

[18] Abhiram Kothapalli and Srinath Setty. 2022. SuperNova: Proving Universal Machine Executions without Universal Circuits. IACR Cryptology ePrint Archive, Report 2022/1758. https://eprint.iacr.org/2022/1758

[19] Abhiram Kothapalli and Srinath Setty. 2023. HyperNova: Recursive Arguments for Customizable Constraint Systems. IACR Cryptology ePrint Archive, Report 2023/573. https://eprint.iacr.org/2023/573

[20] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. https://doi.org/10.1109/TDSC.2023.3348760

[21] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1804–1816. https://doi.org/10.14778/3594512.3594513

[22] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2968–2985. https://doi.org/10.1145/3460120.3485379

[23] Claire Mathieu, Rajmohan Rajaraman, Neal E. Young, and Arman Yousefi. 2024. Competitive Data-Structure Dynamization. *ACM Transactions on Algorithms* 20, 4 (2024).

[24] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2006. An Integrated Efficient Solution for Computing Frequent and Top-k Elements in Data Streams. *ACM Transactions on Database Systems* 31, 3 (2006), 1095–1133. https://doi.org/10.1145/1166074.1166084

[25] OpenVM. 2026. Generating Proofs. https://docs.openvm.dev/book/writing-apps/generating-proofs/ Accessed 2026-05-03.

[26] Charalampos Papamanthou, Shravan Srinivasan, Nicolas Gailly, Ismael Hishon-Rezaizadeh, Andrus Salumets, and Stjepan Golemac. 2024. Reckle Trees: Updatable Merkle Batch Proofs with Applications. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM. https://doi.org/10.1145/3658634.3670354
[27] Polygon Labs. 2026. Polygon zkEVM Proving Architecture. https://docs.polygon.technology/tools/zkevm/architecture/zkprover/stark-recursion/proving-architecture/ Accessed 2026-05-03.
[28] Mayank Rathee, Yuwen Zhang, Henry Corrigan-Gibbs, and Raluca Ada Popa. 2024. Private Analytics via Streaming, Sketching, and Silently Verifiable Proofs. IACR Cryptology ePrint Archive, Report 2024/666. https://eprint.iacr.org/2024/666
[29] RISC Zero. 2026. Recursive Proving Documentation. https://dev.risczero.com/api/recursion Accessed 2026-05-03.
[30] Srinath Setty. 2022. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In *Advances in Cryptology – CRYPTO 2022*. 359–388. https://doi.org/10.1007/978-3-031-15985-5_13
[31] Starknet. 2026. SHARP: Starknet Proof Aggregator Documentation. https://docs.starknet.io/learn/protocol/sharp Accessed 2026-05-03.
[32] Succinct. 2026. SP1 Proof Aggregation Documentation. https://docs.succinct.xyz/docs/sp1/writing-programs/proof-aggregation Accessed 2026-05-03.
[33] Peter Todd. 2016. Merkle Mountain Ranges. OpenTimestamps technical specification. https://github.com/opentimestamps/opentimestamps-server/blob/master/doc/merkle-mountain-range.md
[34] Zhaoyu Wang, Pingchuan Ma, Zhantong Xue, Yuguang Zhou, Qixin Zhang, Xiaoqin Zhang, and Shuai Wang. 2026. ZK-Value: A Practical Zero-Knowledge System for Verifiable Data Valuation. arXiv preprint arXiv:2605.03581. https://arxiv.org/abs/2605.03581
[35] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *30th USENIX Security Symposium*. 501–518.
[36] Jianzhe Yu, Feng Han, Qi Dong, Qiyao Luo, Weiran Liu, Lin Qu, and Ke Yi. 2026. Sketch-Based Secure Query Processing for Streaming Data. *Proceedings of the ACM on Management of Data* 4, 3, Article 233 (2026). https://doi.org/10.1145/3802110

A Framework Guarantees

The framework's guarantees follow by composing local transition soundness with recursive history soundness.

Local transition soundness. For every closed epoch i, the per-epoch transition proof π_i is knowledge-sound for $\mathcal{R}^{\tau}_{\mathsf{upd}}$ on public input $(D_i, \mathsf{Com}(S_{i-1}), C_{B_i}, \mathsf{Com}(S_i), \mathsf{M})$. Therefore, an accepting proof binds $\mathsf{Com}(S_i)$ to $\mathsf{Com}(S_{i-1})$ and C_{B_i} under the declared batch update, and pins the epoch contents to the upload-log slice declared by D_i via the SliceOK predicate. A prover who fabricates the new state, omits a record from the committed epoch, substitutes the epoch contents away from the upload-log slice, or applies the wrong sketch algorithm cannot produce an accepting proof except with negligible probability.

History soundness. For every interval $[i, j]$, the recursive proof $\Pi_{[i,j]}$ verifies the child interval proofs and enforces equality of adjacent boundary state commitments. By induction on the interval tree, the root proof $\Pi_{[1,t]}$ certifies that $\mathsf{Com}(S_t)$ is reachable from $\mathsf{Com}(S_0)$ by applying the committed epochs $B_1, \ldots, B_t$ in order, with their declared descriptors $D_1, \ldots, D_t$ tiling the upload-log prefix without gaps or overlaps.

Query-prefix soundness. For a query at logical time j, the prover first closes all uploads before j into a committed epoch and proves the resulting history root for version $t = \mathrm{ver}(j)$. The decode proof then certifies the decoded value *dec* against $\mathsf{Com}(S_t)$, and (when the release policy is non-trivial) the release proof certifies $ans = policy(dec; \rho)$. Hence an accepting query proof is tied to exactly the upload-log prefix visible at logical time j, in line with Property 3 of §3.5.

Tamper resistance. Dropping an epoch, reordering two epochs, replacing an epoch commitment C_{B_i}, substituting a descriptor D_i, or substituting an intermediate state changes either a leaf public input, an interval boundary, or the ordered-epoch digest $H_{[1,t]}$ (which now binds $(D_1, C_{B_1}), \ldots, (D_t, C_{B_t})$). Under commitment binding and hash collision resistance, such a change cannot produce the same root commitment $R^{(t)}$ with an accepting root proof, except with negligible probability.

Succinct verification. The verifier checks only the root proof $\Pi^{(t)}$ against the root commitment $R^{(t)}$, the current state commitment $\mathsf{Com}(S_t)$, and the public metadata. Verification cost is independent of the number of updates in the verification window and independent of the number of records inside those updates.

Privacy. Zero knowledge hides the update witnesses, including raw records, sketch states, delta tables, sorted certificates, and commitment openings. Circuit obliviousness ensures that proof shape depends only on public capacities and parameters. Thus, the verifier learns the declared metadata, the committed history root, and the validity of the certified statements, but not the private records or private intermediate states.

B Design Space and Cost Derivations

This appendix has three roles: (i) it walks the plaintext Count-Min update inside ZK and quantifies the per-cost-axis pains on the running example (the *warm-up*, formerly §4.1 in earlier drafts); (ii) it names and quantifies the `naive-zkSketch` strawman as the cost to beat (formerly §4.2); and (iii) it places committed sketches relative to exact ZK indexes through a compact cost comparison.

Warm-up: plaintext Count-Min inside ZK.. Count-Min's plaintext update is a one-liner on a record (k, v): compute d keyed hashes $h_i(k)$ and increment d cells $S[i, h_i(k)]$ += v. Inside a ZK circuit this becomes a sequence of d in-circuit algebraic-hash evaluations ($\sim$ 300 constraints each), d private-address Merkle paths ($\sim 2 \log w \times 300 \approx 4{,}800$ constraints per record at $\log w = 8$), within-batch collision sequencing, and a state recommit. On a production deployment ($m = 1024, d = 7, w = 2^{16}, H \approx 300$) the direct encoding costs $\sim 3.4 \times 10^7$ constraints per batch — an order of magnitude above the optimised pipeline of §4 + §5. Three challenges crystallise the gap: **C1** private random writes at hash-determined addresses; **C2** within-batch collisions resolved in data-dependent order; **C3** capacity-dependent leakage (touched-cell count is a function of key skew). The framework's delta certificate addresses C1 by certifying the net effect rather than the procedure; a committed public hash profile addresses C2 by replacing in-circuit hashing with committed-lookup evaluation; and public capacity bounds in M address C3 by padding every witness to a data-independent shape.

Strawman: `naive-zkSketch` *and the cost to beat.* We call the direct encoding above `naive-zkSketch`. It pays for three things the actual sketch update does not need to certify: per-record per-row Merkle paths ($\Theta(md)$), in-circuit hashing ($\Theta(mdH)$), and full-state commitment recompute ($\Theta(s)$). The delta-certificate construction of §5 replaces all three: the prover groups writes outside the circuit and certifies the grouping by permutation/sortedness/segment aggregation ($\Theta(md)$ rows verified, not paths opened); the committed public

hash profile replaces in-circuit hashing with committed lookups ($\Theta(mdH) \to \Theta(mdL)$, $L \ll H$); the homomorphic-additive vector commitment reduces the recommit to the touched cells ($\Theta(T)$ instead of $\Theta(s)$). For the running parameters with $T = 64$ and $L \approx 30$, the post-optimisation per-batch cost is around 4×10^5 constraints — an $\sim 80\times$ saving over `naive-zkSketch`. Per-family circuit-level optimisations stack further on top when their preconditions hold: hash-once-per-distinct-key (Appendix C.2) cuts hash invocations from $|B_t|$ to the distinct-key count $u_{\max}$ under skewed workloads (4–8× on Zipf-α = 1 streams), and wide-output two-field hashing (Appendix C.2) halves per-key hash invocations on Pattern A families. We do not implement `naive-zkSketch` as an evaluation column in §6; it is a rhetorical cost benchmark that makes the framework's structural savings explicit.

Comparative cost analysis. The rest of this appendix places committed sketches relative to exact ZK indexes through a compact cost comparison in arithmetic-circuit constraint counts and derives every entry of Table 5 from a per-row arithmetic-circuit accounting.

Setting. A committed dataset D of n records evolves through batches $B_1, B_2, \ldots$ of m records each. We compare an *exact index* (sorted-index commitment with lookup arguments, in the methodology of PoneglyphDB and ZKSQL [16, 21]) against a *committed sketch* maintained by the delta-certified design of §4. Each strategy admits a naive arithmetic-circuit encoding (replay the plaintext computation) and an optimized one. H denotes per-Poseidon-evaluation cost; L denotes per-lookup-argument cost.

Table 5: Asymptotic arithmetic-circuit cost per query / per single-record update for naive and optimized baselines, ignoring lower-order constants. Boldface entries highlight where each baseline wins.

	Naive		Optimized	
Operation	Index	Sketch	Index	Sketch
Point query	$\Theta(H \log n)$	$\Theta(dH \log w)$	$\Theta(L)$	$\Theta(dL)$
Range freq.	$\Theta(RH \log n)$	$\Theta(dH \log w)$	$\Theta(RL)$	$\Theta(dL)$
Top-k	$\Theta(nH \log n)$	$\Theta(sH)$	$\Theta(nL)$	$\mathbf{\Theta(sL)}$
Distinct	$\Theta(nH \log n)$	$\Theta(sH)$	$\Theta(nL)$	$\mathbf{\Theta(sL)}$
Quantile	$\Theta(nH \log n)$	$\Theta(s'H)$	$\Theta(nL)$	$\mathbf{\Theta(s'L)}$
Per-rec. update	$\Theta(H \log n)$	$\Theta(dH \log w)$	$\Theta(H \log n)$	$\mathbf{\Theta(d + L)}$

Three observations carry the design space. *(1) Point queries are comparable across optimized baselines*: optimized index pays one lookup; optimized sketch pays d openings. *(2) Aggregate queries asymptotically favor the optimized sketch*: top-k, distinct, quantile, arbitrary-range frequency all run in $\Theta(s \cdot L)$ or $\Theta(s' \cdot L)$ vs. $\Theta(n \cdot L)$ for the index. *(3) Sketches do not universally dominate*: exact-equality tests and per-record retrieval are the index's authority.

Naive arithmetic-circuit baselines. Naive exact index. Per-record update is a $\log n$-deep Merkle path with one Poseidon per level: $\Theta(H \log n)$. Per point query: $\Theta(H \log n)$. Per range-frequency of R keys: $\Theta(RH \log n)$. Per top-k/distinct/quantile: $\Theta(nH \log n)$. *Naive sketch (CM).* Per record: d in-circuit hashes ($\Theta(dH)$) + d Merkle openings ($\Theta(dH \log w)$) + d rebindings + collision sequencing. Total: $\Theta(dH \log w)$. Aggregate decode against the committed sketch state: $\Theta(sH)$, independent of n.

Optimized arithmetic-circuit baselines. Optimized exact index. Per point query: one lookup ($\Theta(L)$); per range: $\Theta(RL)$; per top-k/distinct/quantile: $\Theta(nL)$. Per-record update under authenticated balanced-tree maintenance: $\Theta(H \log n)$. *Optimized sketch.* Delta-certified maintenance gives per-record update $\Theta(d + L)$ amortized; aggregate queries cost $\Theta(sL)$ or $\Theta(s'L)$.

Streaming regime. The per-record update ratio between optimized exact index and optimized sketch is $\Theta(H \log n/(d+L))$, which grows with n for any fixed d and reasonable H, L. At $H \approx 200$, $L \approx 5$, $d = 7$, $n = 2^{30}$, the ratio is ≈ 500. The streaming sketch-maintenance regime separates the two strategies on every per-update workload regardless of query mix.

C HyperLogLog (Pattern A specialization)

HyperLogLog (HLL) is a Pattern A sketch with a max-based update operator. The committed state is an array of s small registers

$$S_{\mathsf{HLL}}[b] \in \{0, \ldots, \ell\}, \qquad b \in [s].$$

Each uploaded record is a key k. The declared hash profile contains two hash functions:

$$h_1 : \mathcal{K} \to [s], \qquad h_2 : \mathcal{K} \to \{0,1\}^{\ell}.$$

For a key k, the plaintext update computes

$$b = h_1(k), \qquad \rho = \min\big(\mathsf{LeadingOnes}(h_2(k)) + 1,\ \rho_{\max}\big),$$

and updates the register by

$$S_{\mathsf{HLL}}[b] \leftarrow \max\{S_{\mathsf{HLL}}[b], \rho\}.$$

The exact convention for LeadingOnes (zero- or one-indexed, all-one-string handling) and the saturation cap $\rho_{\max} \le \ell$ are fixed by the public metadata M. The circuit range-checks every ρ value against $[0, \rho_{\max}]$ and every stored register value against $[0, \ell]$.

C.1 Update Circuit

The HLL transition circuit certifies one committed epoch

$$B_t = \{k_j\}_{j=1}^{m}$$

against an old state commitment C_{old} and a new state commitment C_{new}. The batch effect is a sparse register-wise maximum patch: one row per touched register, carrying the largest ρ-value contributed by the epoch to that register.

HyperLogLog Update Circuit.
Public input. Epoch descriptor D_t, batch commitment C_{B_t}, old state commitment C_{old}, new state commitment C_{new}, and metadata M fixing s, ℓ, the hash profile, register representation, estimator convention, commitment scheme, and public capacities.
Private witness. Batch table B, generated-register table U, register-sorted table $U^{\uparrow}$, sparse max-patch table A, and old/new opening tables O, N.
Output. A proof that C_{new} commits to the result of applying the declared HLL batch update to C_{old} using the committed batch C_{B_t}.

Verification target. For every register $b \in [s]$,

$$S_t[b] = \max\left(S_{t-1}[b], \max_{j:\, h_1(k_j)=b} \mathsf{LeadingOnes}(h_2(k_j))\right).$$

Step 1: Bind the private batch to the public commitment. The circuit starts with a private batch table

$$B[j] = (a_j, k_j),$$

where $a_j \in \{0,1\}$ is an active flag. It verifies that active rows of B open from C_{B_t}, and that inactive rows are canonical dummy rows.

Step 2: Generate register contributions. For each active row $B[j] = (1, k_j)$, the circuit computes

$$b_j = h_1(k_j), \qquad y_j = h_2(k_j), \qquad \rho_j = \mathsf{LeadingOnes}(y_j),$$

and checks

$$0 \le b_j < s, \qquad 0 \le \rho_j \le \ell.$$

The generated-register table is

$$U[j] = (a_j, b_j, \rho_j).$$

Inactive batch rows produce canonical dummy rows in U.

Step 3: Bring contributions to the same register together. The prover supplies a register-sorted table $U^{\uparrow}$. The circuit verifies

$$U^{\uparrow} \equiv U$$

as multisets over the semantic columns (a, b, ρ), and checks that $U^{\uparrow}$ is sorted by register id b, with inactive rows placed last. After this check, all contributions to the same HLL register form one contiguous segment.

Step 4: Aggregate by register using max. The circuit performs segment aggregation over $U^{\uparrow}$. For each active segment with register id b, it computes

$$M_b = \max_{j:\, h_1(k_j)=b} \rho_j.$$

The output is a sparse max-patch table

$$A[z] = (\beta_z, b_z, M_{b_z}),$$

where $\beta_z \in \{0,1\}$ is an active flag. Each active row of A corresponds to one HLL register touched by the epoch. The circuit additionally checks that the active rows of A are sorted by register id b, contain no duplicate active register ids (each touched register appears at most once), and that inactive rows are canonical dummies; this normal form is what lets the state-rebinding check of Step 5 treat the max-patch as a well-defined sparse map.

Step 5: Apply the sparse max patch. The prover supplies old and new opening tables O and N for the registers listed in A. For every active patch row $(b, M_b) \in A$, the circuit checks

$$N[b] = \max\{O[b], M_b\}.$$

It also verifies that $O[b]$ opens correctly against C_{old}, that $N[b]$ opens correctly against C_{new}, and that all opened register values are in the declared range.

Unlike Count-Min, HLL cannot use an additive commitment relation of the form $C_{\mathsf{new}} = C_{\mathsf{old}} + \mathsf{Com}(\Delta)$, because the update operator is max rather than addition. The state-rebinding check is therefore mandatory and must bind every register — not only the touched ones — by one of the three backend-supported mechanisms: (i) a sparse-max-update vector-commitment proof that explicitly binds untouched coordinates as equal to C_{old}; (ii) batched old/new openings of the touched registers together with an explicit equality-outside-the-patch argument over all untouched indices (e.g. via a permutation/lookup that pairs each untouched $O[b']$ with the corresponding $N[b']$ and checks equality); or (iii) a whole-state replacement proof when sparse max updates are not supported by the chosen commitment. The equality-outside-the-patch requirement is what prevents the classic sparse-opening attack of mutating untouched registers, and is required for the same reason the Count-Min homomorphic-additive relation is required in Step 7 of §5.3.

Step 6: Check metadata compatibility and expose the history leaf. Finally, the circuit checks that M fixes the HLL parameters, hash profile, register representation, estimator convention, commitment scheme, and public capacities. The resulting transition proof is exposed as the leaf payload consumed by the recursive history-composition layer of §4.3.

Update summary. The HLL transition circuit proves four claims: the batch rows are bound to C_{B_t}; each active key produces the declared register contribution $(h_1(k), \mathsf{LeadingOnes}(h_2(k)))$; contributions to the same register are aggregated by maximum; and the new committed state is obtained by applying the resulting sparse max patch to the old committed state.

C.2 Update-circuit optimisations

The default Update Circuit above is the canonical HLL transition. Three optimisations apply on top of it. All three are intrinsic to the Pattern A max-update relation rather than backend-specific, and all three preserve the verification target of the gatebox: each optimisation changes the witness layout the prover supplies, not what the circuit certifies.

Optimisation A: Hash once per distinct key. Step 2 of the default circuit invokes the declared hash on *every* active batch row. Because HLL's primary workload is distinct counting, the batch typically contains many duplicates of the same key, so this is wasteful: the contribution of any duplicated key is determined by its hash alone, and that hash need only be evaluated once per distinct key. The optimisation pre-groups the batch by key before Step 2. Concretely, the prover supplies a key-grouped table

$$G_{\mathsf{HLL}}[u] = (\alpha_u, k_u), \qquad u \in [U_{\max}],$$

of public capacity $U_{\max}$ (declared in M), and the circuit verifies $\pi_k(G_{\mathsf{HLL}}) \equiv \pi_k(B)$ as multisets of *distinct* keys — either by a sort-and-segment deduplication of B by key with a $\#\{\text{distinct keys}\} = \#G_{\mathsf{HLL}}$ witness, or by a multiplicity-discarding multiset-equality argument over the key column. Multiplicities are dropped because HLL is order- and multiplicity-insensitive: only the set of distinct keys in the epoch matters. Step 2 then hashes each row of G_{HLL} once. Under a Zipfian or otherwise duplicate-heavy workload this is the largest hash-cost reduction available to HLL: the per-batch hash count drops from $|B_t|$ to $u_{\max} \le \min(|B_t|, s)$, and the saving is highest in exactly the regime where HLL is the natural choice (high-cardinality distinct counting on streams whose duplication rate is, by definition of the application, large).

Optimisation B: One wide-output hash with a two-field split. The default circuit declares two hash functions h_1 and h_2 and invokes them separately. In practice both outputs can be derived from a single wide-output algebraic-hash permutation by reading disjoint output limbs of one committed digest: the low $\log_2 s$ bits become the register index b, and the next ℓ bits feed the LeadingOnes computation. The hash profile records the split (which limbs map to b and which to the LeadingOnes input), the bit widths of the two fields, and the LeadingOnes convention; the circuit verifies the range constraints $0 \le b < s$ and $0 \le \rho \le \ell$ exactly as in the default Step 2. This optimisation halves the per-(distinct-)key hash count without changing soundness, because both fields are deterministic functions of a single committed hash output and the split is part of the public hash profile. It composes with Optimisation A: under both, the per-batch hash invocation count drops from $2|B_t|$ permutations (default) to $u_{\max}$ permutations.

Optimisation C: Sparse register-rebind commitment. The state-rebinding check of Step 5 must bind every register, but mechanism (i) of Step 5 (the sparse-max-update vector-commitment proof with explicit equality-outside-the-patch) is the natural fit: most epochs touch far fewer than s registers, and the cost surface should scale with the touched-register count rather than with s. Under mechanism (i), the per-batch commitment work is $\Theta(T_{\max} + \log s)$, where $T_{\max}$ is the public touched-register capacity declared in M (with $T_{\max} \le \min(u_{\max}, s)$); the additive $\Theta(\log s)$ accounts for the standard path-style bookkeeping that the chosen vector-commitment scheme uses to certify which registers are unchanged. Mechanism (ii) (batched openings of the touched registers plus a permutation-based equality-outside-the-patch argument over all untouched indices) has the same asymptotic profile and is preferable when the vector-commitment scheme exposes a multi-opening primitive rather than a sparse-max primitive directly. Mechanism (iii) (whole-state replacement) should be used only when neither (i) nor (ii) is available, because its cost is $\Theta(s)$ regardless of how many registers the epoch actually touched.

Composing the three optimisations. Under all three, the per-batch update circuit cost decomposes into $\Theta(|B_t|)$ batch-bind rows, $\Theta(|B_t|)$ key-deduplication rows, $u_{\max}$ wide-output hash invocations, $\Theta(u_{\max})$ register-aggregation rows, and $\Theta(T_{\max} + \log s)$ commitment-rebind rows. Two structural costs remain. The first is the linear scan over the committed batch: every uploaded record must be bound to C_{B_t}, so this term is inherent to the upload log of §3.3 and cannot be amortised away by any update-circuit optimisation. The second is the small range checks on ρ values: their bit width is $\lceil \log_2(\ell + 1) \rceil$ (typically ≤ 6 bits for any practical HLL precision), so they are second-order to the hash and commitment terms in every operating regime and inherent to the max-update relation rather than slack.

C.3 Query Circuit

An HLL query is parameter-free: it estimates the number of distinct keys from the committed register array. In zkSketch, queries are answered against a closed sketch version, so the query circuit reads only the committed HLL state.

The HLL decode circuit implements the estimator declared in M using fixed-point arithmetic and lookup tables for the powers 2^{-R} and the bias-correction constants α_s (and any small-/large-range correction the declared variant uses); the fixed-point precision β (bit width of the mantissa) and the lookup-table contents are public metadata and the circuit constraint shape depends only on s and β. The prover opens the register array $S_t[1], \ldots, S_t[s]$ from the latest state commitment and the circuit verifies the openings; two estimator modes are supported below.

HLL estimator mode (in-circuit cardinality). The circuit computes

$$Z = \sum_{b=1}^{s} \mathsf{Pow2Neg}[S_t[b]],$$

where $\mathsf{Pow2Neg} : \{0, \ldots, \ell\} \to \mathbb{Q}_\beta$ is the public fixed-point lookup table of 2^{-x} at precision β, and then certifies the cardinality

$$dec = \alpha_s \cdot s^2 \cdot \mathsf{Inv}_\beta(Z)$$

in fixed-point form, where Inv_β is the declared fixed-point reciprocal (computed by a lookup-checked multiplication $\mathsf{Inv}_\beta(Z) \cdot Z = 1$ to precision β). When the release policy of §3.3 is the identity, $ans = dec$ is published in the declared fixed-point encoding; when *policy* is non-trivial (e.g. integer rounding to the published cardinality estimate, or noise addition from a committed seed ρ), *dec* remains a private witness and the proof certifies $ans = policy(dec; \rho)$ against the committed state version, in line with the dec/ans split of Property 3 of §3.5.

LogLog score mode (raw statistic). For backends where the fixed-point reciprocal of estimator mode is too expensive, the circuit instead computes the raw LogLog-style score

$$R = \sum_{b=1}^{s} S_t[b],$$

and treats R as the decoded value: $dec = R$. The declared estimator $\widehat{D} = \alpha'_s s\, 2^{R/s}$ becomes part of the release policy and is certified by the release proof when *policy* maps *dec* to $\widehat{D}$; under this mode the in-circuit cost shrinks at the price of moving the bias-correction lookup out of the decode circuit. Identity release publishes $ans = dec = R$ directly; non-trivial release publishes $ans = policy(dec; \rho)$ with *dec* kept as a private witness, so R is never published in a mode where the policy is meant to round, suppress, or privatise the answer. Either way the public output of the circuit is *ans*, not R.

HyperLogLog Query Circuit.
Public input. State commitment C_{state}, metadata M (which fixes the estimator mode, the fixed-point precision β, and the release policy), and published answer *ans*.
Private witness. Opened register array $S_t[1], \ldots, S_t[s]$, opening witnesses, the decoded value *dec* (the fixed-point cardinality $\alpha_s s^2 \mathsf{Inv}_\beta(Z)$ in estimator mode, or the raw score R in LogLog mode), the intermediate estimator values $Z, \mathsf{Inv}_\beta(Z)$ in estimator mode, and the committed release-seed opening ρ when *policy* is non-trivial.
Verification target. The opened registers are the committed HLL state; in estimator mode the fixed-point estimator computation gives $dec = \alpha_s s^2 \mathsf{Inv}_\beta(Z)$; in LogLog mode $dec = R$ via the declared summation; and *ans* satisfies either $ans = dec$ (identity policy) or $ans = policy(dec; \rho)$ (non-trivial policy).

Checks. State openings, register range constraints, fixed-point Pow2Neg lookup against M, fixed-point Inv_β relation (estimator mode), declared summation (LogLog mode), and the identity / release-policy binding from *dec* to *ans*.

D SpaceSaving (Pattern B specialization)

We instantiate SpaceSaving as a *declared bounded-retention variant*: the circuit certifies the deterministic top-s summary specified in M, rather than replaying the online eviction-with-error-fields procedure of textbook SpaceSaving [24] step-by-step. This variant is appropriate for the streaming-sketch framework because it captures the heavy-hitter retention behaviour while being deterministic and capacity-bounded; deployments that require the textbook online eviction with per-row overestimate error fields would need an additional eviction-error gate beyond what is certified here. This appendix assumes *nonnegative* increments. Delete-aware heavy-hitter maintenance requires a different declared variant and is outside this SpaceSaving instantiation; if a workload needs signed/negative updates, use the signed Count-Min of §5.3 instead. The committed state stores at most s active key–count pairs

$$S_{\mathrm{SS}}[z] = (a_z, k_z, c_z), \qquad z \in [s],$$

where $a_z \in \{0,1\}$ is an active flag. Active rows are sorted by a public total order

$$(c,k) \succ (c',k') \iff c > c' \text{ or } (c = c' \text{ and } k \prec_{\mathsf{M}} k'),$$

where $\prec_{\mathsf{M}}$ is the public deterministic tie-breaker on keys fixed in M (e.g. lexicographic on the key encoding). Inactive rows are canonical dummies and appear after active rows.

The declared batch update follows the batch merge rule: union the old summary rows with the new epoch rows, group by key and sum counts, then retain the top $\min(s, |G|)$ key–count rows under the public order (so if fewer than s candidates exist, exactly $|G|$ active rows are kept and the rest are canonical dummies). This is a Pattern B certificate because the update is a global summary replacement, not a sparse update to fixed cells.

D.1 Update Circuit

The SpaceSaving transition circuit certifies one committed epoch

$$B_t = \{k_j\}_{j=1}^{m}$$

against an old state commitment C_{old} and a new state commitment C_{new}. If the workload uses weighted updates, each batch row is (k_j, v_j); the unweighted case is recovered by setting $v_j = 1$.

SpaceSaving Update Circuit.
Public input. Epoch descriptor D_t, batch commitment C_{B_t}, old state commitment C_{old}, new state commitment C_{new}, and metadata M fixing the capacity s, key representation, count representation, tie-breaking order, commitment scheme, and public table capacities.
Private witness. Batch table B, old summary table S_{old}, union table U, key-sorted union table $U^{\uparrow k}$, grouped candidate table G, kept table S_{new}, residual table R, and commitment-opening witnesses.

Output. A proof that C_{new} commits to the result of applying the declared SpaceSaving batch update to C_{old} using the committed batch C_{B_t}.
Verification target. Let

$$W_k = \sum_{r:\, S_{\text{old}}[r].k=k} S_{\text{old}}[r].c + \sum_{j:\, k_j=k} v_j.$$

The new committed state contains the top s active key–count pairs (k, W_k) under the declared count/key order, padded with canonical dummy rows if fewer than s candidates exist.

Step 1: Bind the old state and the private batch. The circuit verifies that the private old summary table

$$S_{\text{old}}[r] = (a_r, k_r, c_r)$$

opens from C_{old}, is sorted by the declared count/key order, has distinct active keys, and uses canonical dummy rows after the active prefix. It also verifies that the private batch table

$$B[j] = (a_j, k_j, v_j)$$

opens from C_{B_t}, with $v_j = 1$ in the unweighted setting and canonical dummy rows for inactive entries.

Step 2: Form the union table. The prover supplies a union table U containing all active old summary rows and all active batch rows:

$$U = \{(k_r, c_r, \text{old}) : S_{\text{old}}[r] \text{ active}\} \cup \{(k_j, v_j, \text{new}) : B[j] \text{ active}\}.$$

The circuit verifies that U is exactly this multiset union, including active flags and source tags. Source tags are not part of the logical summary; they are only used to bind rows to either the old state or the new committed epoch.

Step 3: Group by key and sum counts. The prover supplies a key-sorted table $U^{\uparrow k}$. The circuit checks

$$U^{\uparrow k} \equiv U$$

as multisets and verifies that $U^{\uparrow k}$ is sorted by key, with inactive rows last. It then checks segment aggregation over equal-key runs, producing a grouped candidate table

$$G[p] = (\gamma_p, k_p, W_p),$$

where

$$W_p = \sum_{r:\, S_{\text{old}}[r].k=k_p} S_{\text{old}}[r].c + \sum_{j:\, k_j=k_p} v_j.$$

Thus G contains one active row per distinct key appearing either in the old summary or in the new epoch.

Step 4: Certify top-s retention. The prover supplies two tables: a kept table S_{new} of capacity s, and a residual table R. The circuit verifies the conservation relation

$$G \equiv S_{\text{new}} \uplus R$$

as multisets over active flags, keys, and counts. It verifies that S_{new} is *prefix-active* (all active rows come before inactive dummy rows), is sorted by the declared count/key order on its active prefix, has distinct active keys, and exposes the exact-kept-count invariant

$$\#\{z : S_{\text{new}}[z] \text{ active}\} = \min(s, |G|),$$

so that exactly s active rows are kept when at least s grouped candidates exist, and exactly $|G|$ active rows are kept otherwise (with the remaining slots being canonical dummies).

Let τ be the last active row of S_{new}, or a public dummy threshold if fewer than s candidates exist. The circuit checks the threshold-residual condition

$$R[q] \preceq \tau$$

for every active residual row $R[q]$, where $\preceq$ is the non-strict version of the declared order (which uses the public $\prec_{\mathsf{M}}$ tie-breaker on keys). This proves that no omitted candidate is better than the worst retained candidate. Together with the exact-kept-count invariant and the deterministic tie-breaker, the kept table is the canonical top-$\min(s, |G|)$ subset of G.

Step 5: Recommit the bounded summary. The circuit verifies that the kept table opens to the new state commitment C_{new}. This is a whole-state replacement check: the SpaceSaving state is a bounded ordered summary, and its correctness is a global top-s property rather than a per-cell patch.

Step 6: Check metadata compatibility and expose the history leaf. Finally, the circuit checks that M fixes the declared SpaceSaving variant, capacity s, key representation, count representation, ordering rule, commitment scheme, and public table capacities. The resulting transition proof is exposed as the leaf payload consumed by the recursive history-composition layer of §4.3.

Update summary. The SpaceSaving transition circuit proves four claims: the old summary and epoch rows are bound to their public commitments; the candidate table correctly groups old and new contributions by key; the new summary contains the top s grouped candidates under the declared order; and the new state commitment binds to this kept table. This is the Pattern B analogue of batch-effect certification: the certificate proves a valid bounded summary replacement rather than a sparse numerical patch.

D.2 Update-circuit optimisations

The default Update Circuit above already realises SpaceSaving's top-s retention as a one-pass certificate. This subsection records what is and is not an optimisation lever for SpaceSaving, because the structural opportunities are very different from the other Pattern B family (Greenwald–Khanna of §5.4). Two of the three items below are negative results: they identify candidate optimisations that look attractive by analogy with other families but do not actually apply. Surfacing these explicitly avoids overstating the optimisation surface in the cost summary of §5.5.

Optimisation A: Single-round retention is canonical (do not iterate). SpaceSaving's top-s retention is a single threshold cut: once the grouped candidate table G of Step 3 is in hand, the kept set is the top $\min(s, |G|)$ rows under the declared count/key order, and this is determined by one comparison of every residual row against the threshold τ from the last active kept row. This is structurally different from Greenwald–Khanna's iterative fusion, which alternates parity-rounds of adjacent-pair merges and incurs an unavoidable $\Theta(\log(|G|))$ round count for any deterministic schedule that respects rank monotonicity. In SpaceSaving the analogous quantity is simply 1: the certificate is one pass over G producing $(S_{\text{new}}, R, \tau)$ in a single shot. Some sketches in the heavy-hitters family are stated in textbooks as repeated eviction loops, which can tempt an implementation to pad an "eviction-round count" as a circuit-shape parameter

for uniformity with Greenwald–Khanna. That padding is purely implementation convenience: a faithful SpaceSaving transition is one round, and the circuit should expose a fixed one-round shape rather than a tunable round count. Doing otherwise re-introduces the avoidable logarithmic round factor that distinguishes Pattern B fusion from Pattern B retention.

Optimisation B: One-pass threshold scan with a single comparator per row. Step 4 of the default circuit can be realised with a single comparator-style range check per row of G: for each grouped candidate, verify either that it is among the kept rows (placed in S_{new} in the declared order) or that its count/key pair satisfies $(c_p, k_p) \preceq \tau$. Concretely, after the permutation argument certifies $G \equiv S_{\text{new}} \uplus R$, the residual condition $R[q] \preceq \tau$ is a range-style inequality on the count column (with a deterministic tie-breaker on the key column when counts collide), and the kept-table sortedness is the same inequality applied to adjacent rows of S_{new}. Both are decomposable into a sign bit plus a range check on the count difference, with bit width $\lceil \log_2 c_{\max} \rceil$ where $c_{\max}$ is the count-field bit width declared in M. Comparators are second-order to the permutation argument because they sit on bounded-width columns, not on the hash or commitment data path. This is why SpaceSaving avoids the comparator-cost concern that surfaces in the analogous Pattern B fusion certificate: comparators here are cheap by construction and there are exactly $|G|$ of them, not $|G|$ times a round count.

Optimisation C: The whole-state recommit is an inherent floor, not slack. Step 5 binds the entire s-row kept table to C_{new}. A natural attempt to optimise this is to commit only to the *changed* rows of the summary (the SpaceSaving analogue of the sparse-patch commitment of Pattern A). This attempt does not pay off, and the reason is structural rather than incidental. In a heavy-hitter workload the new epoch typically reshuffles a large fraction of the summary — old heavy hitters may be displaced, new ones may be inserted, counts move many keys across each other in the count/key order, and the ordering itself is part of the committed state. The number of changed rows is in expectation $\Theta(s)$, not $o(s)$, so a sparse-rebind commitment carries no asymptotic advantage even when the underlying scheme supports it. The committed summary is also small by design (capacity s, not the much larger dimension that an HLL register array or a Count-Min counter table has), so the absolute cost of the whole-state recommit is modest and the linear $\Theta(s)$ term is the inherent floor for a Pattern B bounded-retention sketch. Surfacing this honestly — in contrast with Pattern A, where sparse openings do reduce the commitment-rebind cost asymptotically — prevents the cost analysis from claiming a lever that the algorithmics of top-s retention does not actually expose.

Composing the three observations. Under Optimisation A (single-round retention) and Optimisation B (one-pass comparator scan), the per-batch update circuit cost decomposes into $\Theta(|B_t| + s)$ bind/union/permutation rows, $\Theta(|G|)$ comparator-style range checks on the count column, and $\Theta(s)$ commitment-rebind rows from Optimisation C's inherent floor. This is single-pass in $|B_t| + s$ with no hidden round multiplier, which is the structural reason that SpaceSaving's circuit cost surface is the cleanest of the four families covered by this paper: there is no logarithmic round factor (unlike Pattern B fusion), no hash cost on the per-batch update path (unlike Pattern A array sketches that hash every distinct key), and no sparse-versus-flat commitment trade-off to tune (the recommit cost is fixed at $\Theta(s)$ by the algorithm, not by the backend).

D.3 Query Circuit

A SpaceSaving top-c query returns the first c active rows of the committed summary under the declared order. Since zkSketch answers queries against closed sketch versions, the query circuit reads only the committed state S_t. If a deployment keeps an explicit unmerged buffer, the buffer must be merged into the summary before answering a top-c query.

SpaceSaving Top-c Query Circuit.
Public input. State commitment C_{state}, query size c, metadata M (which fixes the release policy and tie-breaker $\prec_{\mathsf{M}}$), and published answer table *ans*.
Private witness. Opened SpaceSaving table S_t, opening witnesses, the decoded top-c table *dec*, and (when the release policy is non-trivial) the opening of the committed release seed ρ.
Verification target. The opened table is the committed SpaceSaving state, is sorted by the declared count/key order on its active prefix, and the decoded top-c table satisfies

$$dec[z] = S_t[z], \qquad z = 1, \ldots, \min(c, \#\text{active}(S_t)),$$

with canonical dummy rows in positions $z > \min(c, \#\text{active}(S_t))$. For identity release policy, the circuit additionally proves $ans = dec$; for non-trivial $policy$ (e.g. suppression below a count threshold, k-anonymity collapse, or noise from ρ), *dec* remains a private witness and the circuit proves $ans = policy(dec;\ \rho)$.
Checks. State opening, active-prefix structure, distinct-key condition, sortedness by the declared count/key order, equality between *dec* and the prefix of the opened state up to $\min(c, \#\text{active}(S_t))$, and the identity or release-policy binding from *dec* to *ans*.

Point-frequency query. A point query for a key q can be supported as an auxiliary decoder. The prover opens the committed SpaceSaving table and supplies equality bits

$$e_r = [S_t[r].k = q].$$

The circuit checks $e_r \in \{0, 1\}$, verifies the equality relation for each row, and computes

$$ans = \sum_{r=1}^{s} e_r \cdot S_t[r].c.$$

Because the update circuit enforces distinct active keys, at most one row contributes. If no row matches q, the answer is 0. This is the declared SpaceSaving estimate for q under the bounded summary.

E Constraint-Count Breakdown

We use the representative deployment $m = 1024$, $d = 7$, $w = 2^{16}$, $T \approx 7000$, $s, s' \le 2^{14}$, and a standard IPA-based vector commitment. The numbers are order-of-magnitude estimates intended to ground relative shares.

Commitment-scheme cost across regimes. Naive Merkle: $\sim T \cdot 2850 \approx 2 \times 10^7$ constraints—larger than every other line item combined. *Homomorphic-additive vector commitment* (Pattern A linear): $T \cdot 150 + O(1) \approx 1.05 \times 10^6$, a 15–30× improvement. *Batched IPA/KZG* (Pattern A max, HLL): $T + \log |S| \approx 7019$ multi-opening ops, in the

Table 8: SpaceSaving update with whole-state recommit ($s = m = 1024$).

Component	Constraints	Share
Buffer union (SS-Union)	30,000	6%
Sort + segment grouping	225,000	44%
Top-s certificate	100,000	20%
ReplaceState (MSM over $\|S\|$)	155,000	30%
Total	$\sim 5.1 \times 10^5$	100%

Table 9: Greenwald–Khanna update with $\log s'$ fusion rounds and whole-state recommit ($s' \approx 9.8 \times 10^4$, $m = 1024$).

Component	Constraints	Share
GK-InsertSort + InitDelta	350,000	23%
FuseTuple ($\lceil \log s' \rceil = 17$ rounds)	720,000	48%
StableCompact	80,000	5%
ReplaceState (MSM over $\|S\|$)	350,000	23%
Total	$\sim 1.5 \times 10^6$	100%

10^5–10^6 range. *Whole-state recommit* (Pattern B): one MSM of size $|S| \le s$ or s'.

Per-family closed-form cost equations. For Count-Min (Pattern A linear under committed-lookup hashing and a homomorphic-additive vector commitment) the per-batch update cost is

$$C_{\text{prv}}^{\text{CM}} = m \cdot L_{\text{lkp}} + 2(md)\log(md) \cdot c_{\text{cmp}} + T \cdot (c_{\text{seg}} + c_{\text{patch}}) + c_{\text{lin}}.$$

For HyperLogLog (Pattern A max under batched IPA/KZG opening),

$$C_{\text{prv}}^{\text{HLL}} = m \cdot (L_{\text{lkp}} + c_{\text{lr}}) + 2m \log m \cdot c_{\text{cmp}} + T \cdot c_{\text{seg}} + (T + \log|S|) \cdot c_{\text{open}} + c_{\text{lin}},$$

where c_{lr} is the per-record LeadingRun gate constant and c_{open} is the batched-multi-opening per-row constant. For SpaceSaving (Pattern B with whole-state recommit on a state of bounded size s),

$$C_{\text{prv}}^{\text{SS}} = (s+m)c_{\text{union}} + 2(s+m)\log(s+m)c_{\text{cmp}} + (s+m)c_{\text{seg}} + sc_{\text{topS}} + sc_{\text{patch}} + c_{\text{lin}},$$

where c_{union} is the per-row SS-Union copy constant and c_{topS} is the threshold-residual per-row constant. For Greenwald–Khanna (Pattern B with $\log s'$ FuseTuple rounds),

$$C_{\text{prv}}^{\text{GK}} = (s'+m)c_{\text{insort}} + (s'+m)\log(s'+m)c_{\text{cmp}} + (s'+m)\log(s')c_{\text{fuse}} + s'c_{\text{patch}} + c_{\text{lin}},$$

where c_{insort} is the GK-InsertSort per-row constant, c_{fuse} is the FuseTuple per-pairwise-decision constant, and the $\log s'$ factor reflects the rounds of pairwise fusion. The constants $\{c_{\text{cmp}}, c_{\text{seg}}, c_{\text{patch}}, c_{\text{lin}}, L_{\text{lkp}}, c_{\text{lr}}, c_{\text{open}}, c_{\text{union}}, c_{\text{topS}}, c_{\text{insort}}, c_{\text{fuse}}\}$ are calibrated against published arithmetic-circuit constraint-count microbenchmarks from prior ZK-database systems and yield the per-family numbers in Tables 6–8 below.

Table 6: Count-Min update with committed-lookup hashing and homomorphic-additive vector commitment.

Component	Constraints	Share
Hash verification (committed-lookup, $md = 7168$)	36,000	2%
Sort / permutation	750,000	40%
Segment aggregation	70,000	4%
Commitment patch	1,050,000	55%
Tree append	1,000	<1%
Total	$\sim 1.9 \times 10^6$	100%

Table 7: HyperLogLog update with batched IPA opening.

Component	Constraints	Share
Hash verification (m Poseidon + LeadingRun)	154,000	27%
Sort / permutation	100,000	18%
Segment max + sparse rebind	10,000	2%
Commitment opening	300,000	53%
Total	$\sim 5.6 \times 10^5$	100%

After applying the recommended schemes, no single component dominates by an order of magnitude; total cost lands in the 0.5–3.0 million constraint range. This is a 10–20× improvement over the naive Merkle baseline.

F Future Extensions

The linear chain, balanced binary tree, and streaming forest of §4.3 cover the binary recursive-SNARK composition design space at the framework level. Three directions remain open. *k-ary aggregation*: replacing the binary merge atom with a k-way merge, so a single recursive proof verifies k child proofs at once. This generalises the balanced binary tree, trades per-merge circuit size for fewer levels, and improves core utilisation when k scales with available parallelism; the analogous extension of the streaming forest replaces the binary-counter cascade with a base-k counter cascade. *Hybrid topologies*: combining the smoothed peak chain (for steady-state inserts) with balanced-tree or k-ary finalisation (for query-burst service over the surviving peaks), so the same forest pays uniform per-insert cost and exposes parallelism on demand. *Adaptive policy*: dynamically choosing among the three peak-chain policies and the smoothing rate c from real-time workload signals — query inter-arrival distribution, observed cascade depth, available merge workers — rather than fixing them at deployment time. We leave careful evaluations of all three to future work.